

Programming in C

Raphael 'kena' Poss

This syllabus covers the part “Programming in C” of the course “Datastructuren” delivered to 1st year BSc INF and 2nd year BSc KI at the University of Amsterdam.

Copyright © 2017, Raphael 'kena' Poss. Permission is granted to distribute, reuse and modify this document according to the terms of the Creative Commons Attribution-ShareAlike 4.0 International License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/4.0/>.

Contents

1 Course introduction

2 Introduction to C by example

2.1 Summary

3 How to speak C aloud

3.1 Summary

4 Printing stuff

4.1 Intro to printf

4.2 Intro to numbers and characters

4.3 Intro to putchar

4.4 Summary

5 Interacting with the environment

5.1 Command-line arguments: argc and argv

5.1.1 Be careful with arrays

5.1.2 Accepting numbers from the command-line

5.1.3 Command-line: summary

5.2 Standard input

5.2.1 Reading character by character

5.2.2 Handling “End of File”

5.3 Summary of all the things

6 Functions

6.1 Function definitions

6.2 Arguments passed by value

6.3 Arguments passed by reference

6.4 Procedures

6.5 Summary

7 What you just learned without me telling

8 Expressions

8.1 Summary

9 Control structures

9.1 Conditional statement (if) and simple loops (while)

9.2 Braces or no braces?

9.3 for loops

9.3.1 for common patterns

9.3.2 for general form and definition

9.4 Summary

10 C's increment and decrement expressions

11 Storing stuff: variables

11.1 Variable definitions

11.2 Variable assignments

12 Definitions and declarations

12.1 Function definitions

12.2 Function declaration

12.3 Header files

12.4 Compilation and linking

12.5 Summary

13 Types

13.1 Basic types

13.2 Composite types

13.3 Array types

13.3.1 Pseudo string type

13.3.2 Commutativity of addition

13.3.3 Summary

- 13.4 Enumeration types
- 13.5 Aggregate types: structures
 - 13.5.1 Structure definitions
 - 13.5.2 Accessing a member of a struct in expressions
 - 13.5.3 Accessing a member of a struct via a reference
 - 13.5.4 Size of a struct in memory
 - 13.5.5 Summary
- 13.6 Aggregate types: unions

14 Objects that outlive functions

- 14.1 References to local variables
- 14.2 Making objects that outlive functions
- 14.3 Releasing objects
- 14.4 What about arrays on the heap
- 14.5 Allocation failures
- 14.6 Summary

15 Control structures (advanced)

- 15.1 First look at `switch`
- 15.2 Control flow viewed as graphs
- 15.3 Intro to `continue` and `break` via the CFG
- 15.4 `for` and its CFG
- 15.5 Constructing arbitrary CFGs
- 15.6 `switch` and its CFG
- 15.7 Summary

16 Expressions (advanced)

- 16.1 Operators, precedence and associativity
- 16.2 Side effects and sequence points
- 16.3 Short-circuit evaluation
- 16.4 Lvalues and rvalues
- 16.5 Note about the array rules
- 16.6 Summary

17 Objects and designators

- 17.1 Objects in C
- 17.2 Storages and lifetimes
- 17.3 Objects vs. designators
 - 17.3.1 Multiple designators
 - 17.3.2 Designators and lifetimes
 - 17.3.3 No designators
 - 17.3.4 Designators of different types for the same object
- 17.4 Object representations
 - 17.4.1 Assignments are just object copies
- 17.5 Linkage of objects, `extern`
 - 17.5.1 How to make an object with external linkage
 - 17.5.2 How to make an object with internal linkage
 - 17.5.3 The double meaning of `extern`
 - 17.5.4 The double meaning of `static`
- 17.6 Two kinds of designators
 - 17.6.1 Mutability
 - 17.6.2 Visibility
- 17.7 Summary so far

18 Pointers, references and designators

- 18.1 References: a logical (abstract) construct
- 18.2 Pointers: mechanism and implementation for references
 - 18.2.1 Pointer types
 - 18.2.2 Pointer values
 - 18.2.3 Pointer rules
 - 18.2.4 Relationship between the pointer rules and array rules
- 18.3 Summary

19 Types, declarations and declarators

- 19.1 Basic and derived types
 - 19.1.1 Operator precedence (reminder)
 - 19.1.2 A simple example
 - 19.1.3 A more complex example
 - 19.1.4 Restrictions on derived types
- 19.2 Declarations and declarators
 - 19.2.1 Declarator positions
 - 19.2.2 `typedef` is about defining type aliases
 - 19.2.3 Abstract declarators
- 19.3 Summary

20 Type and value conversions

- 20.1 Conversion between numeric types
- 20.2 Conversion between pointer types
- 20.3 Mixed pointer / integer conversions
- 20.4 Pseudo-conversions
- 20.5 Invalid conversions

20.6 Automatic and explicit conversions

20.7 Summary

21 Killing the magic in multi-dimensional arrays

22 Machine behavior

22.1 In a nutshell

22.2 Undefined vs Unspecified

22.3 Example unspecified behavior

22.4 Examples undefined behavior

22.5 What really happens upon reaching undefined behavior?

22.6 Further reading

23 Variadic arguments

23.1 Using variadic argument list in a function

24 Pot-pourri of weird things

25 What was not covered in this course

1 Course introduction

- Staff:
 - Teachers: Tim Doolan + Raphael Poss
 - Assistants: Jetske Beks, Marco Brohet, Lorian Coltof, Ysbrand Galama, Devin Hillenius, Erik Kooistra, Kyrian Maat, Thomas Schaper, David Veenstra, Youri Voet, Simon Polstra
- Course page on Datanose:
 - INF: [https://datanose.nl/#course\[56578\]](https://datanose.nl/#course[56578])
 - KI: [https://datanose.nl/#course\[48396\]](https://datanose.nl/#course[48396])
- **Only 1 Blackboard page**
- Lectures:
 - “Datastructuren” every Thursday 3-5pm SP C0.05
 - “Programming in C” - 6 lectures in February - irregular timetable
 - Check Datanose!
- Study work:
 - Labs - 3 groups, 2x2 hours / group / week
 - One assignment per week, posted on Blackboard
 - ~10 hours self study / week
 - Check Datanose! check BlackBoard!
- Grading:
 - INF: 15% PAV, 55% labs, 30% exam
 - KI: 2/3 labs, 1/3 exam
 - Check “Studiewijzer” on Blackboard
- Contact: questions / requests etc
 - Personal stuff: study advisor
 - General stuff: ds2017-assist@list.uva.nl
 - Discussion: IRC channel? Slack?

poll: [prog1 poll link](#) - [results link](#)

- Getting started: you'll need 1) a C compiler 2) a prompt to run your own programs.

Now! Otherwise: <http://codepad.org/>

2 Introduction to C by example

An example program:

```
main() { return 42; }
```

What does this do? Try it!

Steps:

1. Put code into **text file**, name file with extension **.c**
 - Pick your editor: gedit, mousepad, gvim, vim, emacs, etc.
 - Attention: must be “plain text” - no Word, OpenOffice, etc.
2. Compile code, e.g. with command “cc”
 - By default this produces a file named a.out - rename with -o if needed
3. Run program
4. Ask shell for exit status: echo \$?

Demo:

```
$ gedit test1.c
$ cc test1.c      # produces "a.out"!
$ ./a.out
$ echo $?
42
$ cc test1.c -o test1
$ ./test1
$ echo $?
42
```

Let's try another:

```
main() { return 1 + 2 * 3; }
```

What does this program return?

```
$ gedit test2.c # place code in file
$ cc test2.c
$ ./a.out
$ echo $?
(your guess here)
```

Attention: C uses **operator precedence** to determine evaluation order. Like in maths.

Another one (test3.c):

```
main
(
)
{
return
40
+
2
;
}
```

What does this tell you?

The C language is (mostly) insensitive to whitespace.

Let's look at the output of the C compiler a bit closer:

```
$ cc test2.c
test2.c:1:1: warning: return type defaults to 'int' [-Wimplicit-int]
main() { return 42; }
^~~~
```

What's going on?

The C language is very simple, **you must always tell it the type of things.**

Our main function returns an integer number, so we need to explain this (test4.c):

```
// Before: "int" was omitted.
// Now: we place "int" before "main".
int main() { return 42; }
```

Try it! The C compiler should not complain any more.

Oh, and what did we do here? Those two lines:

```
// Before: "int" was omitted.
// Now: we place "int" before "main".
```

The C code can contain **comments**.

A comment in C is enclosed either by // ... [newline], or /* ... */.

Use comments to explain your code! Always.

2.1 Summary

- The smallest program in C starts with "int main() {" and ends with "}"
- **Expressions** order operations using precedence, like in math.
- A program must **return a value** to the environment with "return".
- Whitespace does not matter.
- Comments are ignored by the C language, but matter to the human reader.

3 How to speak C aloud

You must learn how to read C code aloud to someone else, so they can write what you say. You must also learn to listen to someone speaking C to you and write down what they say.

For this you need words:

C thing	English	Nederlands
{ }	brace	accolade
()	parenthesis	parentese, rond haakje
;	semicolon	puntkomma
	space	spatie
	newline	nieuw regel

Don't forget to say "open" and "close" ("gesloten") when needed.

Try it:

```
int main() {  
    return 42;  
}
```

You will need some more words to read most simple C programs:

C thing	English	Nederlands
#	hash, pound	hekje
< >	lower than, greater than	kleiner dan, groter dan
"	double quote	dubbel aanhalingsteken
\	backslash	backslash
'	single quote	enkel aanhalingsteken, apostrof
.	dot, period	punt
,	comma	komma
[]	bracket	haakje, vierkant haakje
/	slash	slash, schuine streep
-	dash, minus	streepje, minteken
_	underscore	underscore, laag streepje
	pipe, vertical bar	pipe, sluitsteken
*	star	sterretje
&	ampersand, "and"	ampersand, "en"
->	arrow, "min greater than"	pijlte, "streepje groter dan"
!=	different from	anders dan, ongelijk aan

With this you can read e.g. the following aloud:

```
#include <stdio.h>  
int main() {  
    printf("hello, world.\n");  
    return 0;  
}
```

The remaining words are somewhat less common but also needed later, so learn them:

C thing	English	Nederlands
^	hat	dakje
:	colon	dubbelepunt
!	bang, exclamation mark	uitroepteken, oproepteken
?	question mark	vraagteken

3.1 Summary

- C uses a lot of punctuation - you'll learn all of it incrementally.
- Most punctuation signs are called the same as when you study literature.
- Different words for different grouping: **braces**, **brackets**, **parentheses** (accolades, vierkante haakjes, parentheses).
- Special cases:

- "*" is "star" ("sterretje"), not "asterisk"
- "|" is "pipe"
- "->" is called "arrow" / "pijlte"
- "!=" is called "different from" / "anders dan"
- "&" is more often called "and" ("en") than "ampersand"

4 Printing stuff

Consider the example from above:

```
#include <stdio.h>
int main() {
    printf("hello, world.\n");
    return 0;
}
```

Try it! What does it do?

Another one:

```
#include <stdio.h>
int main() {
    printf("number: %d\n", 40 / 2);
    return 0;
}
```

Try it! What does it do?

4.1 Intro to printf

printf is defined like this:

```
int printf(string fmt, ...);
```

Which means it takes a string as first argument, then a variable number of arbitrary arguments.

What it does: use the first "format" argument as **template** to print the rest of the arguments.

Example use	Example output
printf("hello");	hello
printf("hel %d lo", 123);	hel 123 lo
printf("%s, hello", "jane");	jane, hello
printf("hello %s", "jane");	hello jane

Note how %d and %s are used:

- %d for **integer numbers**
- %s for **strings**

Summary:

- printf takes always at least one argument. This first argument must be a string.
- For every % in the format, printf takes the next remaining argument and prints it.
- Use %s for strings, %d for numbers.
- If you want a newline at the end of the output, you need to tell printf explicitly with "\n".

4.2 Intro to numbers and characters

What does this program do?

```
int main() {
    printf("hello %d\n", 'i');
    return 0;
}
```

Try it!

poll: prog3 [poll link](#) [poll results](#)

What's going on?

- The special syntax 'i' in the program is translated to the numeric value of the character "i". See the [ASCII table](#) for a correspondance.
- The function printf prints this number with %d as usual.

What does this program do?

```
int main() {
    printf("hello %c\n", 105);
    return 0;
}
```

Try it!

```
$ gedit test5.c
$ cc test5.c
$ ./a.out
hello i
```

What's going on?

The format code `%c` **prints a character with the code given** in the next number argument.

The following three programs print **the same thing**:

```
int main() {
    printf("hello %c\n", 105);
    return 0;
}
```

```
int main() {
    int mychar = 'i';
    printf("hello %c\n", mychar);
    return 0;
}
```

```
int main() {
    printf("hello %c\n", 'i');
    return 0;
}
```

4.3 Intro to `putchar`

The function `putchar` is defined as follows:

```
// Print the character with code c.
int putchar(int c);
```

Another example program:

```
#include <stdio.h>
int main() {
    putchar('h');
    putchar('i');
    putchar('!');
    putchar('\n');
    return 0;
}
```

What do you think?

Now something slightly more complicated:

```
#include <stdio.h>
int main() {
    putchar('h');
    putchar(105);
    return 0;
}
```

What output do you see from this program when you run it from the command line?

poll: prog4 [poll link - poll results](#)

What's going on: the shell is confused when the program does not terminate its output with a newline character.

Do not forget the final newline character.

4.4 Summary

- `printf(fmt, ...)` to print stuff with a format (template).
 - `%d` for numbers,

- %s for strings,
- %c for characters from a code.
- putchar(c) to print individual characters from their code.
- The syntax 'X' (with single quotes!) translates the character “X” to a number automatically.
- Don't forget to print a newline character (\n) at the end of output.
- Use "#include <stdio.h>" at the beginning if you want to use printf or putchar without warnings.

5 Interacting with the environment

You already know how to *output* stuff from a program:

- using the return value;
- using printf to output text.

Now, how does input work?

There are many mechanisms for input, we will use just two of them:

- **command-line arguments**
- **standard input**

5.1 Command-line arguments: argc and argv

To accept command-line arguments your program must start like this:

```
// New: argc, argv
int main(int argc, string argv[]) {
...
}
```

Let's try it out:

```
#include <stdio.h>
int main(int argc, string argv[]) {
    printf("%d\n", argc);
    return 0;
}
```

Run it (you will need to download this file [easy.h](#)):

```
$ gedit test6.c # put code in file
$ cc test6.c -include easy.h
$ ./a.out
1
$ ./a.out a
2
$ ./a.out a a a a a
6
$
```

What's going on?

The special variable **argc** automatically receives the number of arguments, plus one. (Why “plus one”? See below.)

Let's try out some more:

```
#include <stdio.h>
int main(int argc, string argv[]) {
    printf("%s\n", argv[0]);
    return 0;
}
```

```
$ gedit test7.c # put code in file
$ cc test7.c -include easy.h
$ ./a.out
./a.out
$
```

What just happened? Hmmm.

Let's try rename the program file generated by cc:

```
$ mv a.out sometest
$ ./sometest
./sometest

$ mv sometest yahoo
$ ./yahoo
./yahoo
```


What's going on?

The first element of the `argv` array automatically receives the name of the program.

So, `argv` is an “array.” What's that?

An array is a sequence of values next to each other in memory.

In C the elements of an array are always numbered starting from 0.

So what is in `argv` in the 2nd position? Let's find out.

```
#include <stdio.h>
int main(int argc, string argv[]) {
    printf("%s\n", argv[1]);
    return 0;
}
```

```
$ gedit test8.c # put code in file
$ cc test8.c -include easy.h
$ ./a.out hello
hello
$ ./a.out world
world
```

So `argv` automatically contains the command-line arguments.

5.1.1 Be careful with arrays

What happens if we try to use an element that does not exist?

```
$ ./a.out # no extra argument!
Segmentation fault (core dumped)
$
```

Woah. What's going on?

Trying to use an array element that does not exist is BAD.



Don't look at invalid array positions!

Don't write a program that accesses array elements that do not exist!

In the example above this is easy to check: the variable `argc` tells how many elements there are.

Try this:

```
#include <stdio.h>
int main(int argc, string argv[]) {
    // Check the number of arguments given.
    if (argc != 2) {
        printf("woops!\n");
        return 1;
    }

    // Do stuff.
    printf("%s\n", argv[1]);
    return 0;
}
```

```
$ gedit test9.c # put code in file
$ cc test9.c -include easy.h
$ ./a.out
woops!
$ ./a.out hello
hello
```

Easy? Here you've used `if ... else ...` to make a choice. We'll see more about that in a moment.

Now, let's *print all the things!*

```
int main(int argc, string argv[]) {
    for (int i = 1; i < argc; i = i + 1) {
        printf("%s\n", argv[i]);
    }
    return 0;
}
```

```
$ gedit test10.c # put code in file
$ cc test10.c -include easy.h
$ ./a.out hello world
```

What does this program print?

poll: [progc5](#) [poll link](#) [poll results](#)

5.1.2 Accepting numbers from the command-line

`argv` contains strings. What if we want to turn them into numbers to compute with numbers?

Say, we want to make a program that computes the sum of its two arguments.

For this you'll need the following function:

```
// atoi onverts the string argument and produces a number (Ascii To Integer).
int atoi(string v);
```

It's predefined in `stdlib.h`, which you'll need to include. Try this:

```
#include <stdio.h>
int main(int argc, string argv[]) {
    // Check the number of arguments given.
    if (argc != 3) {
        printf("woops!\n");
        return 1;
    }

    // Do stuff.
    return atoi(argv[1]) + atoi(argv[2]);
}
```

```
$ gedit test11.c # put code in file
$ cc test11.c -include easy.h
$ ./a.out 100 23
$ echo $?
123
$
```

You can also try this: (you'll need [easy.h](#))

```
#include <stdio.h>
int main(int argc, string argv[]) {
    // Check the number of arguments given.
    if (argc != 3) {
        printf("woops!\n");
        return 1;
    }

    // Do stuff.
    printf("%d\n", atoi(argv[1]) + atoi(argv[2]));
    return 0;
}
```

```
$ gedit test12.c # put code in file
$ cc test12.c -include easy.h
$ ./a.out 100 23
123
$
```

And maybe you'll find this more readable:

```
#include <stdio.h>
int main(int argc, string argv[]) {
    // Check the number of arguments given.
    if (argc != 3) {
        printf("woops!\n");
        return 1;
    }

    // New: store intermediate results
    // in variables.
    int a = atoi(argv[1]);
    int b = atoi(argv[2]);
    printf("%d\n", a + b);
    return 0;
}
```

5.1.3 Command-line: summary

- use `int main(int argc, string argv[])` to start the program if you want to use command-line arguments.
- `argc` contains the number of arguments plus one (the program name itself).
- `argv` is an array containing the argument strings.
 - starting at position 0 with the program name.
 - last item is at position `argv-1`.
- do not access array elements that do not exist!
- use `atoi` (`#include <stdlib.h>`) to convert a string to a number which you can compute with.

5.2 Standard input

That's the other common way to read input.

It gives access to data that is entered *after the program has already started*.

5.2.1 Reading character by character

You'll use this function:

```
// getchar reads a single character and returns its code.
int getchar();
```

For example:

```
#include <stdio.h>
int main() {
    // Read a character (its code).
    int c = getchar();

    // Print the code.
    printf("hello %d\n", c);
    return 0;
}
```

Run this! What do you see?

```
$ gedit test14.c # put the code in the file
$ cc test14.c
$ ./a.out
(type something here)
```

For example: if you type “i”, your program prints ... 105. If you type “e” your program prints 101.

If you just press the enter key your program prints ... 10, the code for the newline character (see the ASCII table).

Note: the execution of the program is blocked until `getchar()` successfully reads a character. If you do not provide input, your program never terminates.

You can also pass input from another program, for example:

```
$ echo i | ./a.out
hello 105
```

This tells the shell to redirect the output of `echo` to the standard input of your program. This way `echo` provides the character “i” to the standard input, and you do not need to type it in.

5.2.2 Handling “End of File”

Suppose you want to write a program that prints the code of all the characters available on standard input, then stops.

We need a way to know “when there is no more input”.

Conveniently, **getchar()** returns the special value “EOF” when there is no more input available.

Try this:

```
#include <stdio.h>
int main() {
    while(1) {
        // Read a character code.
        int c = getchar();

        // Was this the end of input?
        if (EOF == c) {
            return 0;
        }

        // No, print the character.
        printf("hello %c\n", c);
    }
    return 0;
}
```

```
$ gedit test15.c # put the code in the file
$ cc test15.c
$ echo "world" | ./a.out
hello w
hello o
hello r
hello l
hello d
hello
$
```

What just happened?

- The program contains a loop `while(1) { ... }`
- Inside this loop, the program:
 - reads one character with `getchar()`
 - checks if the End-of-File condition was encountered `EOF == c`
 - if it was encountered, the program terminates with `return`
 - then it prints the character
 - and the loop executes again.

Note: **This general program structure (a loop containing code to read, test, compute, print something repeatedly) is very common. It is called an “event loop” or sometimes “read-eval-print-loop” (REPL).**

5.3 Summary of all the things

- Read from command-line arguments with: `int main(int argc, string argv[])`
 - Be careful with array sizes.
- Read from standard input with `getchar()` and `EOF`.
 - General structure “`while(1) { read ... test ... print }`” is very common.

6 Functions

You've seen plenty of functions already!

`main`, `printf`, `putchar`, `getchar`, `atoi` are all functions.

You can make your own functions too!

6.1 Function definitions

You can make your own functions!

```
// square returns the value of its argument, squared.
int square(int x) {
    x = x * x;
    return x;
}

int main() {
    int c = square(1);
    printf("%d\n%d\n", c, square(2));
    for (int i = 3; i <= 10; i++)
        printf("%d\n", square(i));
    return 0;
}
```

```
$ gedit test18.c # put the code in the file
$ cc test18.c
$ ./a.out
1
4
9
...
89
100
```

6.2 Arguments passed by value

In general a function receives a *copy* of its arguments: the variables in the caller context are not modified!

Consider the following:

```
int square(int x) {
    x = x * x;
    return x;
}

int main() {
    int x = 2;
    int y = square(x);
    x = x + 1;
    printf("%d %d\n", x, y);
    return 0;
}
```

What does this program print?

poll: [prog9 poll link - poll results](#)

What's going on? In square the variable `x` is initialized with a *copy* of `x` in main.

The variable named "x" in square and the variable named "x" in main are two different variables!

6.3 Arguments passed by reference

Sometimes, you really need a function to be able to modify the original variable in its caller.

You can do this as follows:

```
#include <stdio.h>

// this function copies its first argument to the 2nd (writing in
// the caller's context) if it is non-zero, then returns the
// value of the first argument.
int copy_if_nonzero(int x, ref_to(int) y) {
    if (x != 0)
        deref(y) = x;

    return x;
}

int main() {
    int z = 42;

    copy_if_nonzero(0, ref(z));
    printf("%d\n", z);

    copy_if_nonzero(12, ref(z));
    printf("%d\n", z);

    return 0;
}
```

Try with: (you'll need [easy.h](#))

```
$ gedit test20.c # put code in file
$ cc test20.c -include easy.h
$ ./a.out
42
12
```

What's going on?

- "ref_to(int) y" declares the parameter `y` to be "a reference to another variable of type `int`"
- the operator `deref(y)` says "access the variable referenced to by `y`"
- the operator `ref(z)` (in main) says "create a reference to the variable `z`"

What does the following program print?

```
#include <stdio.h>

int square_inplace(ref_to(int) a) {
    int n = deref(a) * deref(a);
    deref(a) = n;
}

int main() {
    int x = 2;
    square_inplace(ref(x));

    ref_to(int) y = ref(x);
    int z = deref(y);

    square_inplace(ref(x));
    square_inplace(ref(z));

    printf("%d %d %d\n", x, deref(y), z);
    return 0;
}
```

poll: progca poll link poll results

What's going on?

1. `int x = 2` : makes a real variable named `x`, initialized to 2;
2. `square_inplace(ref(x))` : `x` updated to 4;
3. `ref_to(int) y = ref(x)` : `y` becomes another name for the variable already named `x`;
4. `int z = deref(y)` : makes another real variable named `z`, initialized to the current value of the variable named `y` (also named `x`), i.e. 4;
5. `square_inplace(ref(x))` : `x` updated to 16;
6. `square_inplace(ref(z))` : `z` (separate from `x`!) updated to 16;
7. `printf`: current value of `x` is 16; current value of `deref(y)` is the same as `x`, 16 too; current value of `z` is 16.

6.4 Procedures

What if you want to define a function *that has no return value*? For example a function that only prints stuff and does not compute.

This is a special case:

use the void return type to define *procedures* (functions without a result).

For example:

```
#include <stdio.h>

// New! "void"
// sayhello greets the person whose name is given as argument.
void sayhello(string name) {
    printf("hello, %s!\n", name);
    // No return value in function with "void" return type.
}

int main(int argc, string argv[]) {
    sayhello("joe"); sayhello("jane");
    if (argc == 2)
        sayhello(argv[1]);
    return 0;
}
```

Try it with: (you'll need [easy.h](#))

```
$ gedit test19.c # put code in file
$ cc test19.c -include easy.h
$ ./a.out
hello joe!
hello jane!
$ ./a.out robert
hello joe!
hello jane!
hello robert!
$
```

6.5 Summary

- you can make your own functions! Syntax: `int foo(...) { ... }`
- return values with `return` — this also stops execution in the function!
- by default, *arguments passed by value*.
- you can *pass arguments by reference*:
 - using `ref_to(T)` to declare a reference parameter,

- `deref(r)` to access the variable referenced to by `r`, and
- `ref(x)` to make a new reference to `x`.
- This syntax needs [easy.h](#)!
- make procedures (no return value) by replacing the return type by "void"

7 What you just learned without me telling

You can **compute stuff** in C using *expressions*.

You can **decide and do stuff** using *control structures*.

You can **store** stuff using *variables*.

You can **abstract** stuff using *functions*.

You can “access variables remotely” with *references*.

You already know all this from the examples!

Now, let's bring some structure to it.

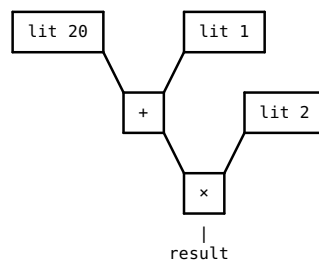
8 Expressions

Expressions in the program are evaluated to a **value** during execution.

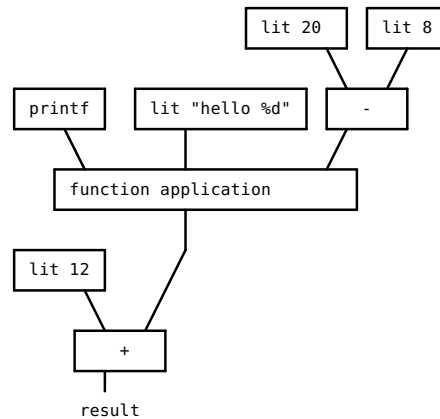
Category	Format	What we call it in source code	Its value during execution
literal	"hello"	a string literal	the string
literal	'x'	a character literal	the character's code (e.g. 'x' has value 88)
literal	123	a number literal	the number
arithmetic	a + b	"a plus b"	the sum of the values of a and b
arithmetic	a * b	"a times b"	the product of the values of a and b
comparison	a < b	"a lower than b"	1 if the value of a is smaller than the value of b, 0 otherwise
comparison	a == b	"a test equals to b"	1 if the value of a is equal the value of b, 0 otherwise
comparison	a != b	"a test different from b"	0 if the value of a is equal the value of b, 1 otherwise
indexing	a[b]	"a bracket b close bracket", or "a indexed by b"	the b-th element of a
application	a(b, c)	"a parenthesis b comma c close parenthesis" or "function a applied to arguments b and c"	the result returned by function a called with the value of b and c as arguments
misc	(a)	"a enclosed in parentheses"	the value of a
misc	<code>ref(v)</code> or <code>addr_of(v)</code> (with easy.h)	"reference to v" or "address of v"	a reference to v
misc	<code>deref(r)</code> (with easy.h)	"deref r" or "r, dereferenced"	the variable referenced to by r

Example expressions in C:

```
(20 + 1) * 2
```



```
12 + printf("hello %d", 20 - 8)
```



Some more expressions:

```
a < (3 + b)
```

```
x[y - 123]
```

To predict the value of an expression, imagine the expression tree and evaluate the tree starting from the leaves.

Quiz: what does the following program print?

```
int main() {
    int c = "hello"['f'-'e'];
    printf("%d\n", c - 1);
    return 0;
}
```

poll: [prog6 poll link - poll results](#)

8.1 Summary

- There are many sorts of expressions in C: $a + b$, $a * b$, etc.
- A function call is an expression too: $a(b)$
- Simple values, also called *literals*, are expression too: 'a', "hello", 123
- During execution the program *evaluates* each expression to determine its *value*.
 - Starting at the leaves of the expression tree.

9 Control structures

9.1 Conditional statement (if) and simple loops (while)

These you need to remember:

Syntax	How we call this
if (...) ...	an if statement
if (...) ... else ...	an if..else statement
while (...) ...	a while loop
do ... while (...)	a do..while loop

Their behavior should be transparent if you already know stuff from other programming languages. Otherwise:

- if (a) b

evaluates expression *a*, then runs *b* if the value of *a* is non-zero.

For example:

```
if (argc != 2) {
    printf("woops!\n");
    return 0;
}
```

- if (*a*) *b* else *c*

evaluates expression *a*, then runs *b* if the value of *a* is non-zero, otherwise runs *c*.

For example:

```
if (argc == 2)
    return atoi(argv[1]);
else
    return 42;
```

- while (*a*) *b*

evaluates expression *a*, then runs *b* if the value of *a* is non-zero, then repeats.

For example:

```
while(1) { int c = getchar(); ... }
```

- do *b* while(*a*)

runs *b* once. Then runs while(*a*) *b*.

9.2 Braces or no braces?

This code is valid:

```
if (argc == 2)
    return atoi(argv[1]);
else
    return 42;
```

This code is valid, too:

```
if (argc == 2) {
    return atoi(argv[1]);
} else {
    return 42;
}
```

What does the following print?

```
if (4 == 2 + 1)
    printf("this is ");
    printf("false\n");
```

General rule: **the braces group statements together.**

Also: **Beware! if..else binds tightly.**

What does the following program do?

```
#include <stdio.h>
int main() {
    if (3 == 2 + 1)
        if (4 == 2 + 1)
            printf("a");

    else
        printf("b");

    printf("\n");
    return 0;
}
```

poll: prog7 [poll link - poll results](#)

What's going on?

else binds to the closest if:

```
if (a) if (b) c else d
```

is equivalent to

```
if (a) { if (b) c else d }
```

always, irrespective of spacing!

When in doubt, use braces.

Actually, **use braces always when you're a beginner**. That will prevent mistakes that are hard to find.

9.3 for loops

Remember this:

- **Usually, you can understand for loops easily using a common pattern.**
- **When you can't recognize a common pattern, use the detailed analysis. This always works.**

9.3.1 for common patterns

Common pattern	What this means	Values of x
for (x = A; x < B; ++x) ZZ	repeat ZZ for x from A to B (exclusive)	A, A+1, A+2 ... B-1
for (x = A; x <= B; ++x) ZZ	repeat ZZ for x from A to B (inclusive)	A, A+1, A+2 ... B
for (x = A; x < B; x = x + C) ZZ	repeat ZZ for x from A to B (exclusive) by C increments	A, A+C, A+2C ... (up to and including the last value A+k*C smaller than B)
for (x = A; x > B; --x) ZZ	repeat ZZ for x from A down to B (exclusive)	A, A-1, A-2, ... B+1
for (x = A; x >= B; --x) ZZ	repeat ZZ for x from A down to B (inclusive)	A, A-1, A-2, ... B
for (x = A; x > B; x = x - C) ZZ	repeat ZZ for x from A down to B (exclusive) by C decrements	A, A-C, A-2C, A-3C ...

For example, you've seen this before:

```
#include <stdio.h>
int main(int argc, string argv[]) {
    int i;
    for (i = 1; i < argc; ++i) {
        printf("%s\n", argv[i]);
    }
    return 0;
}
```

Here `for (i = 1; i < argc; ++i)` makes `i` go from 1 to the value of `argc` (exclusive), and executes `printf(...)` each time.

Another one:

```
#include <stdio.h>
int main() {
    int c;
    for (c = 'a'; c <= 'z'; ++c) {
        printf("%c = %d\n", c, c);
    }
    return 0;
}
```

What does this do? Remember, literal characters evaluate to the character code!

```
$ gedit test16.c # put code in file
$ cc test16.c
$ ./a.out
a 97
b 98
c 99
...
y 121
z 122
$
```

9.3.2 for general form and definition

General syntax:

```
for (a ; b ; c) d
```

Behavior:

1. evaluates a.
2. evaluates b.
3. if the value of b is non-zero, then:
 - runs d, then
 - evaluates c, then
 - start again at step 2.

That's all!

You can imagine the following substitution:

```
for ( ◊ ; ◻ ; △ ) ◇
```

is equivalent to:

```
◊;
while (◻) {
    ◇
    △;
}
```

The three expressions are called the *start expression*, the *condition expression* and the *step expression*.

In other words:

```
for ( start; condition; step ) { something; }
```

is equivalent to:

```
start;
while(condition) {
    { something; }
    step;
}
```

Let's try it:

for code	Equivalent to
<pre>for(i = 0; i < 10; ++i) printf("%d\n", i);</pre>	<pre>i = 0; while (i < 10) { printf("%d\n", i); ++i; }</pre>
<pre>for(c = 'z'; c >= 'a'; --c) printf("%c %d\n", c, c);</pre>	<pre>c = 'z'; while (c >= 'a') { printf("%c %d\n", c, c); --c; }</pre>
<pre>for(c = getchar(); c != EOF; c = getchar()) printf("hello %c\n", c);</pre>	<pre>c = getchar(); while (c != EOF) { printf("hello %c\n", c); c = getchar(); }</pre>

9.4 Summary

- if (a) b else c : "if a then b else c"
 - the parentheses are mandatory around the condition!
- "if (x == a) { b }", not "if x == a { b }"

- else binds tightly!

"if (a) if (b) c else d" equivalent to "if (a) { if (b) c else d }"

- while : "repeat as long as the condition is true"
- do .. while : "do once, then repeat as long as the condition is true"
- for (start ; cond ; step) thing : "do start, then repeat thing then step as long as cond is true"

10 C's increment and decrement expressions

Thanks to "for" you have learned about some more expressions!

Category	Format	What we call it in source code	Its value during execution
unary	++a	"plus plus a"	increment a by 1, then return the new value after the increment
unary	--a	"minus minus a"	decrement a by 1, then return the new value after the increment
unary	a++	"a plus plus"	the value of a before it is incremented by 1
unary	a--	"a minus minus"	the value of a before it is decremented by 1

What does the following program print?

```
#include <stdio.h>
int main() {
    int a = 42, b = 69;
    printf("%d %d ", a++, ++b);
    printf("%d %d\n", a, b);
    return 0;
}
```

poll: [prog8 poll link](#) - [poll results](#)

11 Storing stuff: variables

You have seen plenty of variables already!

Here:

```
int a = 40 + 2; // a is a variable!
```

```
for (int i = 0; i < 20; i++) // i is a variable!
    printf("%d\n", i);
```

```
int main(int argc, string argv[]) {
    // argc and argv are variables!
    ...
}
```

11.1 Variable definitions

What's common to all this:

- you *declare* a new variable using the general syntax "type name;"

(the name of a type, a space, then the name of your new variable, followed by a final semicolon).

For example, "int a;" or "string b;".
- you can optionally set an initial value at the same time as the declaration.

For example, "int a = 42;" or "string b = "hello";"
- you can declare multiple variables separated by commas:

For example, "int a, b;"

Or: "int a = 42, b = 69;"
- the arguments to a functions are variables too!

11.2 Variable assignments

You can set a variable to a new value:

- during its declaration, with the syntax "type name = value;" (see above);
- using the **assignment operator**: "name = newvalue" (you've seen this already!);
- using the increment/decrement operators: "name++", etc. (see above too!)

Beware! Do not use a variable before it is initialized.

For example, the following program is invalid:

```
int main() {
    int a;
    return a;
    a = 42;
    return 0;
}
```

```
$ gedit test17.c # put code in file
$ cc test17.c
test17.c: In function 'main':
test17.c:3:11: warning: 'a' is used uninitialized in this function [-Wuninitialized]
    return a;
           ^
```

We can't run this program. Really. Don't try!



Don't use uninitialized variables!

Another example invalid use:

```
#include <stdio.h>
int main() {
    int c;
    while(c != EOF) { // poor kitten!
        c = getchar();
        putchar(c);
    }
    return 0;
}
```

12 Definitions and declarations

- *Definition*: this is a thing, and here is how it works.
- *Declaration*: I promise to the compiler that this thing exists. Somewhere else.

12.1 Function definitions

General syntax:

```
type name(args...) {
    // ... stuff ...
    return value;
    // ... stuff ...
}
```

You read this as: "the *return type*, then the *function name*, then the *parameter definitions* inside parentheses, then the *body* of the function enclosed in braces."

You can use the `return` statement at any point to stop the execution of the function and return the value of the expression to the *caller*.

12.2 Function declaration

General syntax:

```
type name(args...);  
  
// or, optionally:  
extern type name(args...);
```

You read this as “the optional keyword *extern*, then the *return type*, then the *function name*, then the *parameter declarations* inside parentheses, then a semicolon.”

Notes:

- function declarations are optional: the compiler will make one automatically if you use a function that is not declared; however
- **an implicit function declaration will cause a warning** and you should consider this a programming error.
- a function definition is also a declaration.

Always declare your functions if you don't define them in the same source file!

12.3 Header files

Remember the thing “`#include <stdio.h>`”?

What does this mean?

There's no magic: **it tells the compiler to take the file `stdio.h` and copy it inside your program.**

Look at what's in `stdio.h` yourself:

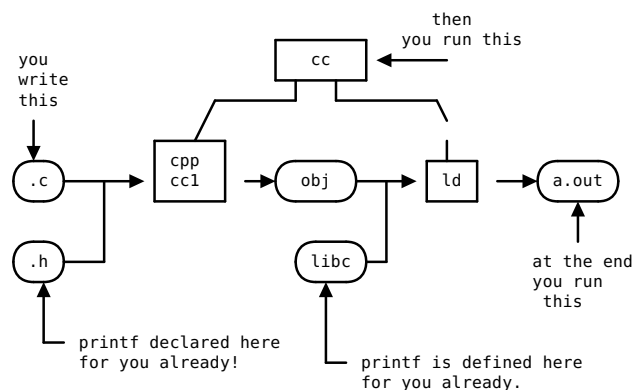
```
extern int printf(string fmt, ...);  
extern int putchar(int);  
extern int getchar();  
#define EOF -1
```

In other words we don't *really* need to use `#include <stdio.h>`, the following works just as well:

```
// Look ma', no #include!  
// We just need a suitable declaration of printf.  
extern int printf(string fmt, ...);  
  
int main() {  
    // just works!  
    printf("hello!\n");  
    return 0;  
}
```

12.4 Compilation and linking

So where does `printf` really come from?



12.5 Summary

- A *definition* makes a function “here and now”.
- A *declaration* promises the function exists somewhere else.
 - If you forget a function declaration, the C compiler makes one for you automatically (with a warning).

- A *header file* is just a bunch of declarations.
- `cc` is a command that really runs three other commands: `cpp`, `cc1` and `ld`.
- `cpp` groups *source code* and *header files* together (and produces source code);-
 - headers are just copy-pasted in the source code where `#include` is used! Nothing more.
- `cc1` *translates* (compiles) source code to *object code*
- `ld` groups object code together with “`libc`”, also called *the C standard library*
 - a *library* is a collection of predefined object files, containing `printf`, `atoi`, `getchar`, etc.

13 Types

So far you have met three types:

- `int` - whole numbers
- `string` - character strings (with [easy.h](#) only!)
- `ref_of(T)` - a reference to another variable of type `T` (with [easy.h](#) only!)

13.1 Basic types

These are the common types: `int`, `char`, `double`.

Here are *all the basic types*:

Name	Synonyms	Description	Size in memory (bytes)	Possible values
_Bool	bool (stdbool.h)	A boolean variable	0 or 1 (always), true or false (stdbool.h)	
char	N/A	A small whole number	$1 \leq S_c$	either $[-2^{S_c-1} \dots 2^{S_c-1} - 1]$ or $[0 \dots 2^{S_c}]$ depends on the environment
signed char	N/A	A small signed whole number	$1 \leq S_c$	always $[-2^{S_c-1} \dots 2^{S_c-1} - 1]$
unsigned char	N/A	A small unsigned (positive) whole number	$1 \leq S_c$	always $[0 \dots 2^{S_c} - 1]$
short	signed short	A smallish whole number	$2 \leq S_s$	$[-2^{S_s-1} \dots 2^{S_s-1} - 1]$
unsigned short	N/A	A smallish positive whole number	$2 \leq S_s$	$[0 \dots 2^{S_s} - 1]$
int	signed int	A whole number	$2 \leq S_i$	$[-2^{S_i-1} \dots 2^{S_i-1} - 1]$
unsigned int	unsigned	A positive whole number	$2 \leq S_i$	$[0 \dots 2^{S_i} - 1]$
long	signed long	A bigger whole number	$4 \leq S_l$	$[-2^{S_l-1} \dots 2^{S_l-1} - 1]$
unsigned long	N/A	A bigger positive whole number	$4 \leq S_l$	$[0 \dots 2^{S_l} - 1]$
long long	signed long long	A yet bigger whole number	$8 \leq S_{ll}$	$[-2^{S_{ll}-1} \dots 2^{S_{ll}-1} - 1]$
unsigned long long	N/A	A yet bigger positive whole number	$8 \leq S_{ll}$	$[0 \dots 2^{S_{ll}} - 1]$
float	N/A	A single precision FP number	4	$[-3.40 \times 10^{38} \dots 3.40 \times 10^{38}] \cup \{-\infty, \infty\}$
double	N/A	A single precision FP number	8	$[-1.80 \times 10^{308} \dots 1.80 \times 10^{308}] \cup \{-\infty, \infty\}$

Guarantee: $S_c \leq S_s \leq S_i \leq S_l \leq S_{ll}$

Some C implementations also provide "long double" but this is non-standard.

For more information about floating-point numbers:
<http://liv.science.uva.nl/dfp.html>

Which type should you use in your own programs?

- if you need a whole number: use int, unless you have a good reason to use something else.
- if you need a floating-point number: use double, unless you have a good reason to use float.
- if you need a character: use char, except when you need an array of characters, then use char.
- if you need a boolean: use bool, possibly int.

Side question: why does C provide so many types if one only needs int, char, double and bool in most programs? That's because these options give control to the programmer over memory usage and performance. Once you start caring

about memory usage and performance, you can start thinking about this again. (note: not in this course.)

13.2 Composite types

Here are *all the composite types*:

- *complex types*: if T is a type, then `"typedef _Complex T t"` defines a new type t :
 - it's the type of a complex number with real and imaginary parts of type T
 - its size in memory is `2 * sizeof(T)`.
- *reference types*: if T is a type, then `"typedef T *t"` (or `"typedef ref_to(T) t"` with [easy.h](#)) defines a new type t :
 - it's the type of a reference to a variable of type T
 - its size in memory depends on the environment: usually 2, 4 or 8 bytes.
- *void reference type*: `"void*"` is a type;
 - it's the type of a reference to a variable of unknown type;
 - its size in memory depends on the environment;
- *array types*: if T is a type, then `"typedef T t[N]"` or `"typedef T t[]"` defines a new type t :
 - it's the type of arrays of values of type T ;
 - its size in memory is either `N * sizeof(T)` if N is specified, or unknown otherwise.
- *function types*: if T, U, V are types, then `"typedef T (*t)(U, V)"` defines a new type t ;
 - it's the type of references to functions taking parameters of type U and V and returning values of type T ;
 - its size in memory is the same as the size of `void*`
- all the *structure types* (see below);
- all the *union types* (see below);
- all the *enumeration types* (see below).

Never forget: <http://cdecl.org/>

Here are some of the standard additional types predefined for you with `typedef` in `stdint.h`:

Name	Description	Size in memory (bytes)	Possible values
<code>int8_t</code>	A 8-bit whole number	1	−128..127
<code>uint8_t</code>	A 8-bit positive whole number	1	0..255
<code>int16_t</code>	A 16-bit whole number	2	−32768..32767
<code>uint16_t</code>	A 16-bit positive whole number	2	0..65535
<code>int32_t</code>	A 32-bit whole number	4	−2147483648..2147483647
<code>uint32_t</code>	A 32-bit positive whole number	4	0..4294967295
<code>int64_t</code>	A 64-bit whole number	8	−9223372036854775808..9223372036854775807
<code>uint64_t</code>	A 64-bit positive whole number	8	0..18446744073709551615
<code>intptr_t</code>	A whole number of size at least as large as <code>sizeof(void*)</code>	(depends)	(depends)
<code>uintptr_t</code>	A positive whole number of size at least as large as <code>sizeof(void*)</code>	(depends)	(depends)
<code>intmax_t</code>	A whole number of the maximum supported size	(depends)	(depends)
<code>uintmax_t</code>	A positive whole number of the maximum supported size	(depends)	(depends)

When are these useful? In this course, probably never. In the course “operating systems”, perhaps once or twice. If you use C professionally, surely.

13.3 Array types

Array types in C are *weird*.

At first they look all right:

```
// Returns the nth element of somearray.
int pick(int n, int somearray[10]) {
    return somearray[n];
}

int myarray[10] = { 3, 2, 1, 4, 5, 7, 1, 3, 4, 5 };

int main() {
    // The function takes an array of 10 ints as parameter;
    // The call gives an array of 10 ints as argument.
    // Everything good.
    return pick(3, myarray);
}
```

Then you figure that the following is correct, too:

```
// Returns the nth element of somearray.
int pick(int n, int somearray[]) {
    return somearray[n];
}

int myarray[10] = { 3, 2, 1, 4, 5, 7, 1, 3, 4, 5 };

int main() {
    // The function takes an array of ints as parameter, size unknown;
    // The call gives an array of 10 ints as argument.
    // This works.
    return pick(3, myarray);
}
```

A function argument can have an array type of unspecified size.

An argument whose array type has a known size **automatically degenerates** to an array type of unknown size.

Or in other words:

- the **definition** of an array variable must specify its size;
- the **declaration** of an array variable can omit its size.

Perhaps not too weird yet, but now go figure this out:

```
// Returns the nth element of somearray.
int pick(int n, int somearray[]) {
    ref_to(int) r = somearray;           // WEIRD
    return r[n];                         // WOOO WEIRD
}

int myarray[10] = { 3, 2, 1, 4, 5, 7, 1, 3, 4, 5 };

int main() {
    ref_to(int) r = myarray; // WEIRD AGAIN
    return pick(3, r); // r is reference, function wants array, HOW IS THIS POSSIBLE?
}
```

Woah.

To understand this, you must absolutely grok four completely **WEIRD** separate mechanisms that happen to work together in this example.

1) **An expression of array type always automatically degenerates to a reference to its first element in every context.**

So this is valid: `int a[10]; ref_to(int) b = a;`

It defines `b` as a reference to the first element of `a`. It is equivalent to: `int a[10]; ref_to(int) b = ref(a[0]);`

This is valid too: `int a[10]; deref(a) = 42;`

It sets the first element of `a` to 42.

2) **It is always valid to give an expression with a reference type to a function that expects an array type.**

So suppose you have a function `int foo(int a[10]) { return a[5]; }`

This is valid: `int b[10]; foo(b);`

This is valid, too: `int b[10]; ref_to(int) c = b; foo(c);`

Now, this is ... dangerous to kittens: `int b[3]; foo(b);`

For ... reasons ... the C compiler *accepts to compile* this last example but *its run-time behavior is undefined* (= kittens are eaten).

3) **The expression `a[b]` is always strictly equivalent to `deref(a+b)` or `*(a+b)`.** This is a pure syntactic equivalence — there is no magic.

4) If **a** is an expression that is a reference to an element of an array, then the expression **a+N** is a reference to the Nth element after the current element.

That's it! It's both devilishly simple and mind-boggedly complex.

Learn these 4 rules by heart.

Now let's deconstruct the example again:

```
int pick(int n, int somearray[]) {
    // here "somearray" has an array type.

    // The following assignment is equivalent to:
    //   ref_to(int) r = ref(somearray[0]); // rule 1
    ref_to(int) r = somearray;

    // The following statement "return r[n]" is equivalent to:
    //   ref_to(int) rn  = r + n;           // rule 3
    //   int            value = deref(rn);    // rule 4
    //   return value;
    return r[n];
}

int main() {
    // In the following call "pick(3, myarray)"
    // The expression "myarray" automatically degenerates:
    // the call is equivalent to:
    //   ref_to(int) r = ref_to(a[0]); // rule 1
    //   pick(3, r);                  // call valid, rule 2
    return pick(3, myarray);
}
```

Some exercises:

poll: [progcc poll link](#) - [poll results](#)

13.3.1 Pseudo string type

Of course the string type does not really exist in C.

In [easy.h](#) you see the following:

```
typedef char* string;
```

What's going on?

Every time you have seen "string" in the previous examples you were really using "ref_to(char)" or "char*".

Meanwhile, all the *string literals* (of the form "... " in code) have type char[], i.e., array of char.

So we have:

```
int printf(ref_to(char), ...);

int main() {
    // The following is equivalent to:
    //   char          s[6] = { 'h', 'e', 'l', 'l', 'o', 0 };
    //   ref_to(char) p  = ref(s[0]); // rule 1
    //   printf(p);
    printf("hello");
    return 0;
}
```

Some exercises:

poll: [progcd poll link](#) - [poll results](#)

13.3.2 Commutativity of addition

Remember, $a + b == b + a$?

Now, rule 3 above says $a[b]$ is strictly equivalent to $deref(a + b)$

So it's also equivalent to $deref(b + a)$

So it's also equivalent to $b[a]$

Wooo! Magic!

What does this program do?

```
int main() {
    printf("hello %c!\n", 1["world"]);
    return 0;
}
```

poll: [progcb](#) [poll link](#) - [poll results](#)

13.3.3 Summary

- Arrays have 4 weird rules, **you must learn them separately**:
 1. expression with “array type” evaluates to reference to first element
 2. a reference is a valid argument for a function that wants an array
 3. `a[b]` equivalent to `deref(a+b) (*(a+b))`
 4. if `a` is a reference to an array element, `a+N` is a reference to the Nth element further
- `string` is actually `ref_to(char) (char*)`
- `a[b] == *(a+b) == *(b+a) == b[a]` (this is weird!)

13.4 Enumeration types

These are boring: the syntax `enum { A, B, C }` is an integer type which can only have the symbolic values `A`, `B` or `C`.

It's possible to also specify numeric values: `enum { A = 12, B = 23, C = 34 }`

Then it's possible to give the entire enumeration a name:

```
typedef enum { A, B, C } myenum;
```

This way one can define more variables with the new type `myenum`. For example:

```
typedef enum { A, B, C } myenum;

int main() {
    int x = A; // valid
    myenum y = B; // valid
    myenum z = 435; // warning (kittens are eaten)

    return x + y + z; // valid: they are just numbers.
}
```

Bonus question: what does the program above print?

poll: [progce](#) [poll link](#) - [poll results](#)

The names used inside the `enum` declaration are called *enumeration constants*.

13.5 Aggregate types: structures

A *structure type* allows you to **logically group multiple variables together**.

Think about a “class” in Java or Python to do something similar.

Why does this matter? Often an application will represent one thing using multiple variable. For example a person can have a name, an age, a gender, an address. If you have more than one persons, you need more variables. Structure types can help organize the code and make it more readable.

So how does this work?

13.5.1 Structure definitions

You define a new structure type like this: `struct { fields... }`

Some examples:

```
// This defines an anonymous struct. Not very useful.
struct {
    char*   name;
    int     age;
    char*   address;
};

// This defines a struct with a name.
struct person {
    char*   name;
    int     age;
    char*   address;
};

// This creates two "objects" with type "struct person".
// The word "struct" is mandatory.
struct person joe = { "joe", 34, "33 elm street" };
struct person jane = { "jane", 23, "1 circle lane" };

// This gives a short name to the struct.
typedef struct person person_t;

// Now we can use the short name.
person_t mark = { "mark", 18, "1 circle lane" };
person_t penelope = { "penelope", 44, "123 61st st" };

```

What's going on?

- Inside "struct { ... }" you declare the *members* of the struct.
- The name after the word "struct" becomes the name of the struct. That name must always be used in combination with "struct", like "struct person", "struct stack", etc.
- You can also make a short name with: `typedef struct structname shortname;`

Some oddities:

- You can define a short name without a struct name:
`typedef struct { ... } shortname;`
- You can define a struct at the same time as one or more variables of its type: `struct person { ... } joe, jane, mark, penelope;`

13.5.2 Accessing a member of a struct in expressions

For example:

```
// Define a struct to represent a person.
struct person {
    char*   name;
    int     age;
    char*   address;
};
// Make a short name.
typedef struct person person_t;

int age_difference(person_t someone, person_t someone_else) {

    // New syntax: x.y
    int diff = someone.age - someone_else.age;

    if (diff < 0)
        diff = -diff;
    return diff;
}

```

General syntax: **if x is an object of struct type, then the expression x.a accesses the member a of x.**

Another example:

```
int main() {
    person_t joe;
    joe.name = "joe";
    joe.age = 34;
    joe.address = "33 elm street";
    printf("name: %s, age: %d, addr: %s\n", joe.name, joe.age, joe.address);
    return 0;
}

```

13.5.3 Accessing a member of a struct via a reference

Say you want to make a function that updates a person record to indicate that person has moved to a new address. The following works (with [easy.h](#)):

```
void move_to(ref_to(person_t) someone, string new_address) {
    printf("person %s moving from %s to %s\n",
        deref(someone).name,
        deref(someone).address,
        new_address);
    deref(someone).address = new_address;
}
```

Now, it just so happens that this kind of code is **very common**. So common, in fact, that the designers of C found it was a good idea to provide a *shorter syntax* to write **equivalent but shorter code**:

```
void move_to(ref_to(person_t) someone, string new_address) {
    printf("person %s moving from %s to %s\n",
        someone->name,
        someone->address,
        new_address);
    someone->address = new_address;
}
```

General definition:

- if **x** is a reference to a struct, then the expression **x->a** accesses the member **a** of the referenced struct.
- (More general): the syntax "**a->b**" is always strictly equivalent to "**deref(a).b**" (or "**(*a).b**").

13.5.4 Size of a struct in memory

The size of a struct (e.g. struct person) can be computed as usual with sizeof: sizeof(struct person)

Can you predict the size of a struct? Usually, but not always accurately. Rule of thumb: **the size of a struct is the sum of the sizes of its members, plus an epsilon**.

(It is not exactly the sum of the size of the members, because of [alignment issues](#). Not important for this course.)

13.5.5 Summary

- structures defined with struct { ... }
- structures can be anonymous; they can have a "long name" or "struct name" (struct blah), or they can have a short name with typedef
- size of struct is sum of sizes of members + epsilon

13.6 Aggregate types: unions

The idea of *union types* in C is an example of an *atavism*: a primitive feature that has become rather useless or inconvenient but not yet weeded out by evolution.

Unfortunately you have to understand them, and even more unfortunately perhaps use them, but know that virtually every language since C has done them better. Eventually you will realize that C's unions are a degenerate, under-equipped ancestor of the much more useful, widely acclaimed [sum types](#), also called [tagged union](#) or [variant in some languages](#).

So, where does this leave us?

General rule: **the C compiler handles union more or less in the same way as struct, except that all the members overlap in memory**.

In effect:

- to define a union use the same syntax as struct, just replace the keyword:
 - anonymous union: union { int a; int b; }
 - named union: union blah { int a; int b; }
 - short name: typedef union blah blih;
- make variables again by replacing the keyword: union blah myvar;
- access members: myvar.a
- access members over a reference: deref(myvar).a OR myvar->a

What's the difference then?

1. **all the members of an union object start at the same place in memory.**
2. **the size of an union is the size of its largest member.**

When do you need to use a union? Not applicable in this course (I think).

14 Objects that outlive functions

Remember what we said about variables in functions:

```
int square(int x) { // this is a first variable "x"
    x = x * x;
    return x;
}

int main() {
    int x = 2; // this is a different variable "x"!
    x = square(x);
    return x;
}
```

Each variable defined in a function **only exists during the execution of that function** — it stops to exist when the function returns.

14.1 References to local variables

Knowing this, what do you think about this:

```
ref_to(int) square(int x) {
    int result = x * x;
    return ref(result);
}

int main() {
    ref_to(int) x = square(2);
    printf("hello %d\n", deref(x));
    return 0;
}
```

poll: [progcf poll link](#) - poll results



Don't return references to local variables!

14.2 Making objects that outlive functions

Say, you really want to create a new variable in a function so that the caller can still use it after the function returns.

For this you need to **allocate the object on the heap**.

The heap is a memory space that is separate from the function-local variables, and **where objects exist until your program explicitly destroys (frees) them**.

How to do this? Like this:

```
#include <stdlib.h>
ref_to(int) square(int x) {

    // This makes an object on the heap and creates
    // a (local) reference to it.
    ref_to(int) result = malloc(sizeof(int));

    // This writes the result in the new object (on the heap).
    deref(result) = x * x;

    // This is now valid:
    return ref(result);
}

int main() {
    ref_to(int) x = square(2);

    // This prints "hello 4".
    printf("hello %d\n", deref(x));
    return 0;
}
```

We are using the function `malloc()` defined like this:


```
// malloc returns a reference to an object of the specified size.
void* malloc(int);
```

Make sure to **give the right allocation size to malloc** — the C compiler will not check this for you.

For example:

```
#include <stdlib.h>
int main() {
    ref_to(int) x = malloc(1);
    deref(x) = 2;
    return deref(x);
}
```

What does this program return?

poll: [progcg](#) [poll link](#) - [poll results](#)



Don't make objects too small with malloc!

14.3 Releasing objects

All objects allocated by `malloc` **remain in the heap until your program gets rid of them.**

How? With this:

```
// The function free releases the object pointed to by the reference given as argument.
void free(void*);
```

So this is correct:

```
#include <stdlib.h>
ref_to(int) square(int x) {

    // This makes an object on the heap and creates
    // a (local) reference to it.
    ref_to(int) result = malloc(sizeof(int));

    // This writes the result in the new object (on the heap).
    deref(result) = x * x;

    // This is now valid:
    return ref(result);
}

int main() {
    ref_to(int) x = square(2);

    // This prints "hello 4".
    printf("hello %d\n", deref(x));

    // Release the object allocated by square.
    free(x);

    return 0;
}
```

Ok and then what about this?

```
#include <stdlib.h>
int main() {
    ref_to(int) x = malloc(sizeof(int));
    deref(x) = 2;
    free(x);
    printf("hello %d\n", deref(x));
    return 0;
}
```



Don't use objects after free()!

Note: same concern with:

```
int main() {
    int *c = malloc(...)
    free(c);
    free(c); // this is also a use after free.
    return 0;
}
```

14.4 What about arrays on the heap

An array is just an object. You can also make an array on the heap!

```
int main() {

    // The following allocates an array of 10 ints on the heap.
    int *p = malloc(10 * sizeof(int));

    p[3] = 42; // valid
    p[0] = 123; // valid
    p[10] = 3; // hmmm (poor kittens!)

    ...

    free(p); // ok
    return 0;
}
```

What's going on? We have declared `p` to be “reference to one int” even though we are allocating an *array*.

Why is that? **It is relatively hard to properly write the type “reference to an array” in C, so that it has become customary to use “reference to the first element” instead.**

And yet it does not matter much, because the array rules (see above) make the syntax `p[N]` valid nevertheless.

If you are curious, here is how one would use the full “reference to an array” notation in C (this is valid but remember, nearly nobody does this in practice):

```
int main() {
    int (*p)[10] = malloc(10 * sizeof(int));

    (*p)[3] = 42; // valid
    (*p)[0] = 123; // valid
    p[3] = 42;    // compile error (hopefully), dead kitten (sometimes)

    free(p); // ok
    return 0;
}
```

14.5 Allocation failures

Try this:

```
#include <stdlib.h>
int main() {
    int *p = malloc(60*1024*1024*1024*sizeof(int));
    for (int i = 0; i < 60*1024*1024*1024; i++) {
        p[i] = 123;
    }
    printf("%d\n", p[100]);
    free(p);
    return 0;
}
```

What does this print?

poll: [progci poll link](#) - [poll results](#)

What's going on?

If `malloc()` fails because “not enough memory”, it will return a special reference value called a “null reference”. This reference always compares true to numeric zero. Your program needs to check this!



Don't access objects via null references!

This is correct:

```
#include <stdlib.h>
int main() {
    int *p = malloc(60*1024*1024*1024*sizeof(int));
    if (p == 0) {
        printf("wooops\n");
        return 1;
    }
    for (i = 0; i < 60*1024*1024*1024; i++) {
        p[i] = 123;
    }
    printf("%d\n", p[100]);
    free(p);
    return 0;
}
```

Bonus exercise:

```
#include <stdlib.h>
int main() {
    int *p = malloc(60*1024*1024*1024*sizeof(int));
    if (p = 0) {
        printf("wooops\n");
        return 1;
    }
    p[100] = 123;
    printf("%d\n", p[100]);
    free(p);
    return 0;
}
```

poll: [progcj poll link](#) - [poll results](#)

What's going on?

```
if (p = 0) // <--- look at this closer!
```

Note: it is often prudent to write `0 == p` instead of `p == 0` to protect against these mistakes: `0 = p` is guaranteed to generate a compile error.

14.6 Summary

- don't use function-local variables after they stop to exist (via a reference)!
- allocate objects on the heap with `malloc()`
 - compute the size to allocate with `sizeof`
- release objects from the heap with `free()`
 - don't use objects after `free()!!`
- allocate arrays with `malloc(N * sizeof(T))` with N number of elements, T type of 1 element.

15 Control structures (advanced)

15.1 First look at `switch`

Say you want to write a function that computes the English language ordinal

suffix: "st", "nd", "rd" or "th" dependong on which number is given as input.

With if it's easy enough:

```
if (number == 1)
    return "th";
else if (number == 2)
    return "nd";
else if (number == 3)
    return "rd";
else
    return "th";
```

This pattern with a lot of if branches that test a single variable can be simplified to the following:

```
switch(number) {
case 1:
    return "st";
    break;
case 2:
    return "nd";
    break;
case 3:
    return "rd";
    break;
default:
    return "th";
    break;
}
```

This is the most common usage pattern of switch.

The general syntax for this pattern is:

```
switch (<variable>) {
case <const1>: <action1>; break;
case <const2>: <action2>; break;
case <const2>: <action2>; break;
...
[ default: <action>; break; ]
}
```

If you need to use the same action for multiple constants, this is possible by simply using multiple case labels for the same action.

For example:

```
switch (<variable>) {
case <const1>:
case <const2>: <common action="">; break;

case <const3>:
case <const4>:
case <const5>: <common action="">; break;

...
}
```

15.2 Control flow viewed as graphs

The most general way to think about control flow in C is to chop the program in *basic blocks*: the longest sub-sequences of statements that are always executed together. Then visualize the control flow as a graph where the basic blocks are nodes and the *possible* control flow branches represented as edges.

For example, the following function has just one basic block:

```
int foo(int x) {
    // The following three things are always executed together.
    int y = x + 2;
    printf("y = %d\n", y);
    return x + y;
}
```

The following function has three basic blocks:

```
int foo(int x) {
    int y = x + 2;
    if (y > 3) {
        return 42;
    }

    return x;
}
```

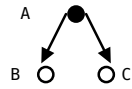
The three basic blocks are:

A) "int y = x + 2; int t = y > 3" (the condition for the if is evaluated at the end of the first basic block).

B) "return 42;"

C) "return x;"

Then we can represent this control flow as follows:



If you remember from earlier, the program that reads from the standard input until it receives EOF:

```
#include <stdio.h>
int main() {
    while(1) {
        // Read a character code.
        int c = getchar();

        // Was this the end of input?
        if (EOF == c) {
            return 0;
        }

        // No, print the character.
        printf("hello %c\n", c);
    }
    return 0;
}
```

This contains the following basic blocks:

A) evaluate 1

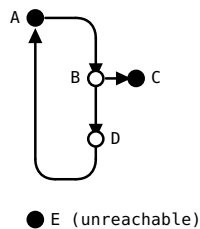
B) read one character, test whether it is EOF.

C) return 0.

D) print the character.

E) return 0 (the 2nd time).

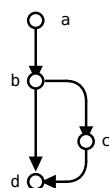
This can be represented as follows:



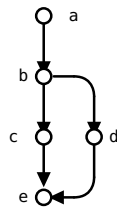
In general, a **loop in a program is equivalent to a control flow graph (CFG) with a cycle.**

Each of the control flow structures in C has its own general, reusable control flow structure:

```
a;
if (b) c;
d;
```



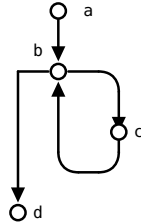
```
a;
if (b) c; else d;
e;
```



```

a;
while (b) c;
d;

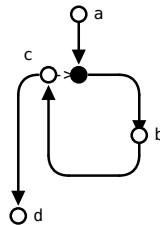
```



```

a;
do b; while (c);
d;

```



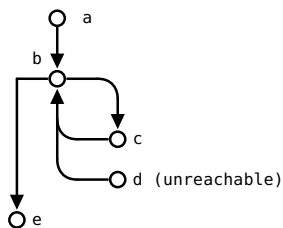
15.3 Intro to `continue` and `break` via the CFG

The special statement `continue` short-circuits the current loop (the innermost loop where it appears) back to the evaluation of the condition.

```

a;
while (b) { c; continue; d; }
e;

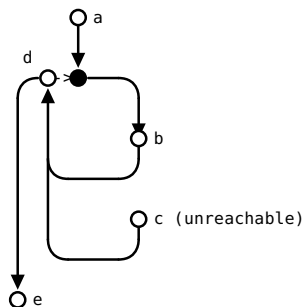
```



```

a;
do { b; continue; c; } while(d);
e;

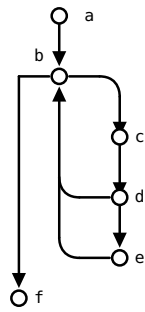
```



```

a;
while (b) { c; if (d) continue; e; }
f;

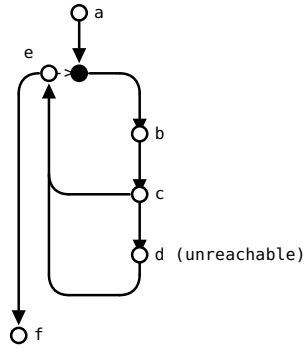
```



```

a;
do { b; if (c) continue; d; } while(e);
f;

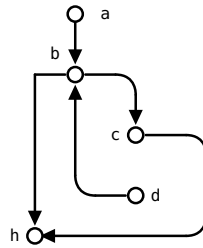
```



```

a;
while (b) { c; break; d }
e;

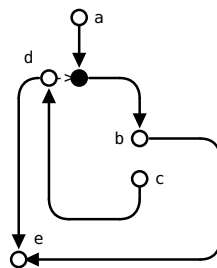
```



```

a;
do { b; break; c; } while(d);
e;

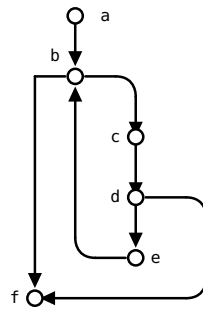
```



```

a;
while (b) { c; if (d) break; e; }
f;

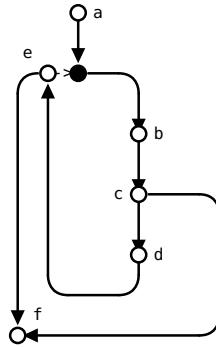
```



```

a;
do { b; if(c) break; d; } while(e);
f;

```



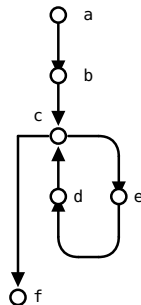
15.4 for and its CFG

Remember, a for loop is a while loop in disguise.

```

a;
for (b; c; d) e;
f;

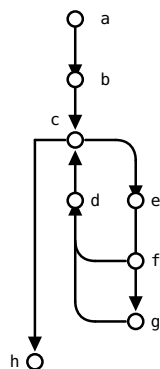
```



```

a;
for (b; c; d) { e; if (f) continue; g; }
h;

```

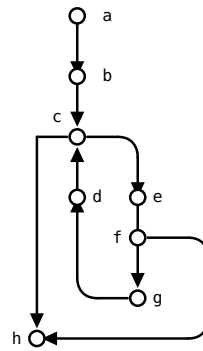


Note: after `continue` in `for`, the loop increment (the `d` block) is still executed!


```

a;
for (b; c; d) { e; if (f) break; g; }
h;

```



15.5 Constructing arbitrary CFGs

The control structures have fixed CFG patterns.

Meanwhile it is possible to construct *arbitrarily complex* CFGs using the goto statement.

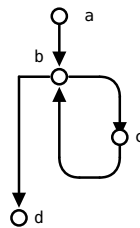
Think about goto like a tool that allows you to draw arbitrary edges in the control flow graph.

For example, take the while loop from above:

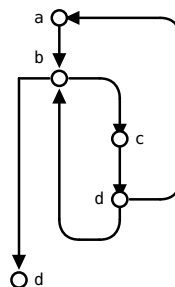
```

a;
while (b) c;
d;

```



Now suppose you want to add a new condition on some expression e after the basic block c, which flows back to the basic block a. That is, you want as end result the following:



Be as it may, there is no standard control structure in C which allows you to reach this result. That's when goto can be applied:

```

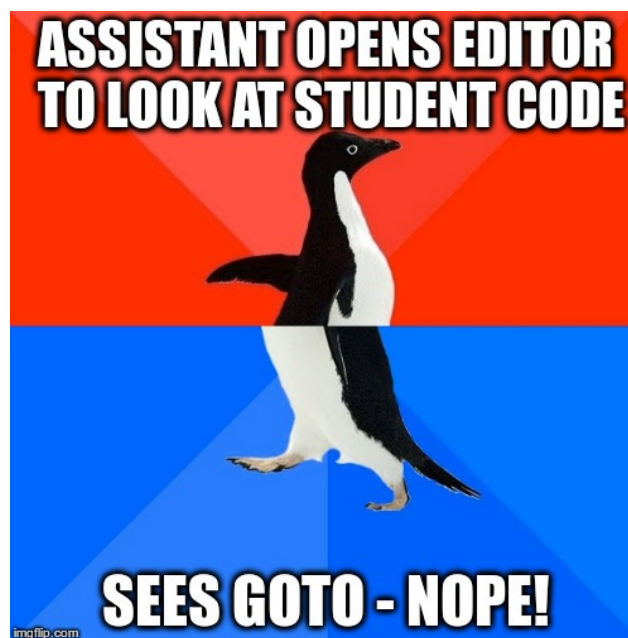
YOURLABEL:
a;
while (b) { c; if (d) goto YOURLABEL; }
d;

```

The way it works is as follows:

- you define a *label* where you want the control flow edge to *arrive*;
- you use goto whenever you want to create a control flow edge that *departs* from the current location towards the label.

Note: using goto in C is (usually) considered Very Bad Style. People will hate you for it, because it makes the code more complex to understand.



goto is very bad style!

However it is handy to learn about it now, because the next section becomes much easier to understand once you understand goto.

15.6 switch and its CFG

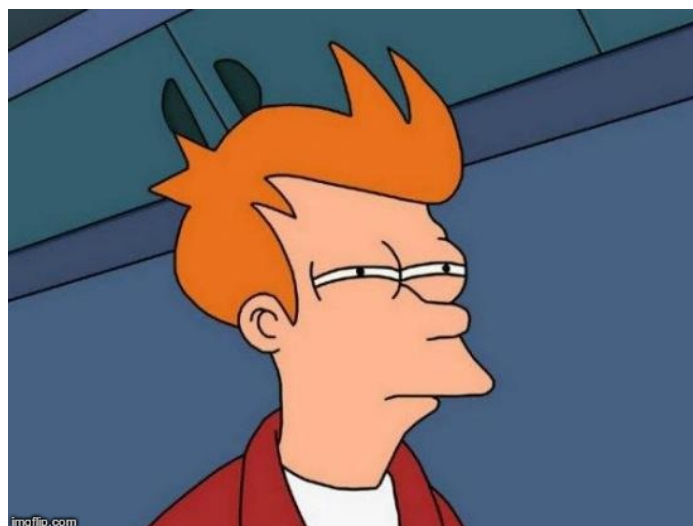
Consider the following code:

```
void duff(int to[], int from[], int count) {
    int n = (count + 3) / 4;
    int i = 0;
    switch (count % 4) {
        case 0: do { to[i] = from[i]; i++;
        case 3:     to[i] = from[i]; i++;
        case 2:     to[i] = from[i]; i++;
        case 1:     to[i] = from[i]; i++;
                } while (--n > 0);
    }
}
```

Is this correct C code?

poll: [progck](#) [poll link](#) - [poll results](#)

Reference: https://en.wikipedia.org/wiki/Duff's_device



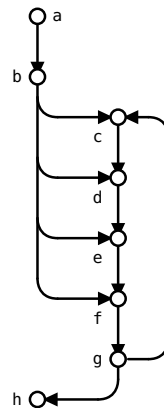
What's going on?

Before explaining further, let's remove the irrelevant syntax and simplify down to the basic block structure:

```

void duff() {
    a;
    switch (b) {
        case 0: do { c;
        case 3:   d;
        case 2:   e;
        case 1:   f;
                  } while (g);
    }
    h;
}

```



What's going on?

To understand this, you need to understand the *general behavior* of `switch`. Actually when you write a `switch` of the following form:

```

a;
switch (b) {
case X:  c;
case Y:  d;
case Z:  e;
}
g;

```

What really gets compiled is something like this:

```

a;

// Conditional part with branching.
if (b == X)    goto L1;
else if (b == Y) goto L2;
else if (b == Z) goto L3;
goto Lend;

// Rest of the code between { ... }
L1: a;
L2: b;
L3: c;

Lend: g;

```

Observation: there is no structural relationship between the `case` labels and the structure of the code inside `switch { ... }`

- the `case` labels are translated to `if` conditions at the beginning, only labels remain;
- the generated labels can appear at arbitrary positions in the `switch { ... }` block.

If the code uses default case, this corresponds to an additional label, with an `else` branch at the beginning:

```

a;
switch (b) {
case X:  c;
case Y:  d;
default: e;
}
g;

```

Translates to:

```

a;

// Conditional part with branching.
if (b == X)    goto L1;
else if (b == Y) goto L2;
else          goto L3; // notice "else" without condition (= default)
goto Lend;

// Rest of the code between { ... }
L1: a;
L2: b;
L3: c;

Lend: g;

```

Also, if you use `break` in `switch`, it's similar to when you use it in a loop: **`break` in `switch` exits to the first basic block after the `switch`.**

It's like replacing "break" by "goto Lend" in the example above.

Now let's come back to the strange thing above:

```

void duff() {
    a;
    switch (b) {
        case 0: do { c;
        case 3:   d;
        case 2:   e;
        case 1:   f;
                } while (g);
    }
    h;
}

```

This is really equivalent to:

```

void duff() {
    a;
    if (b == 0) goto L1;
    else if (b == 3) goto L2;
    else if (b == 2) goto L3;
    else if (b == 1) goto L4;
    goto Lend;

L1:
    do {
        c;
L2:
        d;
L3:
        e;
L4:
        f;
    } while (g);

Lend:
    h;
}

```

Tada!



switch is full of goto in disguise!

15.7 Summary

- imagine control structures as a way to draw a control flow graph (CFG).
 - basic block: group of statements always executed together.
- loops always cause cycles in CFGs.
 - if you want a cycle in the CFG, chances are you want a loop in your code.
- continue goes back to the cycle edge.
 - includes the step expression in for loops!
- break exits the loop to first basic block after the condition.
 - in switch it exits the switch to the first basic block afterwards.
- switch can be seen as a way to simplify a series of ifs
 - but really under the hood it's full of goto.
- try very very very hard not to use goto directly.
 - certainly you won't need it in this course.

16 Expressions (advanced)

16.1 Operators, precedence and associativity

Precedence	Operator	Example	Description	Associativity
1	++ --	a++ a--	<i>Postfix</i> increment / decrement	Left-to-right
	()	a()	Function call	
	[]	a[]	Array subscripting	
	.	a.b	Member access	
	->	a->b	Member access via reference	
	(type){list}	(t){a,b}	Compound literal	
2	++ --	++a --a	<i>Prefix</i> increment / decrement	Right-to-left
	+ -	+a -a	Unary plus and minus	
	! ~	!a ~a	Logical NOT and bitwise NOT	
	(type)	(t) a	Type cast	
	*	*a	Indirection (dereference)	
	&	&a	Address-of	
	sizeof	sizeof a	Size-of	
	_Alignof	_Alignof a	Alignment requirement	
3	* / %	a * b, a / b, a % b	Multiplication, division and remainder	Left-to-right
4	+ -	a + b, a - b	Addition and subtraction	Left-to-right
5	<< >>	a << b, a >> b	Bitwise left and right shift	Left-to-right
6	< <=	a < b, a <= b	Less (or equal) to	Left-to-right
	> >=	a > b, a >= b	Greater (or equal) to	Left-to-right
7	== !=	a == b, a != b	Equal to / not equal to	Left-to-right
8	&	a & b	Bitwise AND	Left-to-right
9	^	a ^ b	Bitwise XOR	Left-to-right
10		a b	Bitwise OR	Left-to-right

11	&&	a && b	Logical AND (with short-circuit evaluation)	Left-to-right
12		a b	Logical OR (with short-circuit evaluation)	Left-to-right
13	?:	a ? b : c	Ternary conditional expression	Right-to-left
14	=	a = b	Simple assignment	Right-to-left
	+= -= *= /= %= <<= >>= &= ^= =	a += b, a -= b, a *= b, etc.	Operation followed by assignment	Right-to-left
15	,	a, b	Comma (sequencing) operator	Left-to-right

Note regarding groups 1 and 2:

- a->b++ is equivalent to (a->b)++ (left-to-right).
- *a++ is equivalent to *(a++) (first postfix ++, then indirection)
- ++a++ is equivalent to ++(a++) (first postfix, then prefix. Also happens to be invalid.).

Note regarding groups 7 and 14: "a = b == c" is equivalent to "a = (b == c)".

Note regarding groups 14 and 15: "a = b, c" is equivalent to "(a = b), c".

16.2 Side effects and sequence points

A **side effect** is when the evaluation of an expression *during execution* “cannot be taken back without someone noticing”.

Example side effects:

- printing something.
- reading something from standard input.
- calling a function that *may* performs side effects.
- (usually) modifying the value of an object.
- modifying a non-local object via a reference.

Example non side effects:

- defining a new variable.
- computing the value of a+2. when a is a simple variable.

A **sequence point** is a point in the code of the program where, during execution, all the side effects of the expressions that precede are guaranteed to have completed, before the expressions that follow start being evaluated.

Example sequence points:

- the semicolon between statements (;).
- the comma operator (,).
- the && and || operators (also, see below).
- the end of a function call expression (after the closing parenthesis in foo(a,b,c)).

Example places **where there is no sequence point**:

- arithmetic operators, e.g. a + b
- the assignment operator, e.g. a = b
- the argument list of a function call, e.g. between a and b in foo(a, b)

What does it mean that “there is no sequence point”? See the definition above: this means *the order of evaluation is not guaranteed*.

Example program:

```
int main() {
    int x = 42;
    printf("%d\n", ++x, ++x);
    return 0;
}
```

What does this program print?

16.3 Short-circuit evaluation

Most of the operators with multiple operands will evaluate all their operands before they compute a result.

There are three exceptions: `&&`, `||` and `?:`.

These operators implement “short-circuit evaluation”: they **first** evaluate their first operand, **then** they test the value of the operand, and **then** they may (or not) evaluate the next operand.

We say there is “hidden control flow” inside them.

In other words:

- “`a && b`” is equivalent to “`r = a; if (r) r = b;`”
- “`a || b`” is equivalent to “`r = a; if (!r) r = b;`”
- “`a ? b : c`” is equivalent to “`if (a) r = b; else r = c;`”

Some exercises:

- what does `printf("a") || printf("b")` print?
- what does `printf("") && printf("b")` print?
- what does `printf("a") ? printf("b") : printf("c")` print?

poll: [progco poll link](#) - [poll results](#)

Note: `printf()` returns the number of characters printed.

16.4 Lvalues and rvalues

Expressions come in two sorts: **lvalues** (“left values”) and **rvalues** (“right values”).

The names comes from the assignment operator (`=`):

- only lvalues can appear on the left of an assignment;
- either lvalues or rvalues can appear on the right of an assignment.

The intuition is (approximately) that “lvalues can be modified, but rvalues cannot.”

In particular the compiler will always produce an error message if the left side of an assignment is not a lvalue.

The operators that *require* an lvalue:

- the assignment operators: `=`, `+=`, `-=`, `*=`, etc.
- the increment/decrement operators: `++`, `--`

How do you get lvalues or rvalues?

- lvalues:
 - the name of a variable: `a` is a lvalue if “`a`” designates a variable.
 - the indirection (dereference) operator: `*a` is a lvalue.
 - the array subscripting operator: `a[b]` is a lvalue (really because `a[b]` is `*(a+b)`).
 - the member access operator: `a.b` and `a->b` are lvalues.
 - a lvalue between parentheses: `(a)` is a lvalue if `a` is one.
 - the assignment operators: `a = b`, `a += b` etc are lvalues (and incidentally the same as `a`).
 - **nothing else.**
- rvalues: **everything else.**

Examples:

- “`++a = 3`” — error: “`++a`” is not a lvalue.
- “`*++a = 3`” — OK: “`*(++a)`” is a lvalue. (assuming `a` is a reference)
- “`foo() = 3`” — error: “`foo()`” is not a lvalue.
- “`foo()[10] = 3`” — OK: “`foo()[10]`” is a lvalue. (assuming `foo()` returns an array or reference to the first element of an array)
- “`int a, b, c; (a == 1 ? b : c) = 42;`” — error: “`?:`” does not produce lvalues.

Why does this matter? So that you can better understand compiler error messages.

16.5 Note about the array rules

Remember the 4 array rules from earlier?

1. an expression with “array type” evaluates to a reference to its first element.
2. a reference is a valid argument for a function that wants an array.
3. `a[b]` equivalent to `*(a+b)`.
4. if `a` is a reference to an array element, `a+N` is a reference to the `N`th element further.

Look at rule 1 more closely. Now look at the following code:

```
int a[10];
int b[10];

a = b;
```

This code is invalid, but why?

- `b` evaluates to a reference to its first element.
- `a` evaluates to a reference to its first element.
- the assignment has two things of the same type on the left and the right (two references), so it should work. Why doesn't it?
- also the error message says “cannot evaluate to expression with array type”. This suggests that the array rule 1 is not applied (or may be wrong). What's up?

That's because the full text of rule 1 is:

1. an expression with “array type” evaluates to a reference to its first element *when it is used as a rvalue*.

Tada!

Now the example `a = b` above has an array on the left (`a` does not degenerate) and a reference on the right (`b` does degenerate) so the assignment is invalid.

(Also, a separate rule in C says that the assignment operators cannot be used on arrays.)

16.6 Summary

- there are *15 precedence levels* in C.
 - mostly intuitive, except perhaps the relative precedence of `=`, `==` and `<`.
 - learn them!
- expressions may have *side effects* (things that happen which cannot be taken back).
 - the order of side effects is determined by expression evaluation order.
 - *sequence point* guarantee when side effects complete, e.g. the semicolon `;`.
 - **there are no sequence points in the argument list of a function call.**
- big difference between *lvalues* and *rvalues* in C
 - only *lvalues* can be assigned to by `=` but also `++` and `--`.
 - first array rule says that array names degenerate to a reference to the first element, but only when they are used as *rvalues*.

17 Objects and designators

This chapter is based on [ISO 9899:2011](#) section 6.2, and “Semantics of C objects”, Appendix F of “[On the realizability of hardware microthreading](#)” by yours truly.

17.1 Objects in C

All the *things* your program manipulates during its execution and that have a *value* and/or *address* are called **objects**.

A “variable” is an object.

A constant is an object.

A function is an object.

A type is *not* an object.

An expression is an object.

A control structure is *not* an object.

Properties of objects:

- its *address*, if it has one (not all objects have an address!)
- its *addressability* — whether it has an address or not (e.g. the constant `"42"`)

- in code is an object at run-time with no address)
- its *mutability* — whether you can change its value or not
- its *size* — number of char's in its representation

17.2 Storages and lifetimes

Definitions:

- **storage**: *where* an object exists in memory.
- **lifetime**: *when* does an object exist (more precisely, *from when* and *until when* — i.e., the two boundaries of a segment of time).

Fundamental semantics of C:

- 1) **The storage of an object determines its lifetime.**
- 2) **The properties of an object are fixed during its lifetime: its mutability, its addressability, its address (if it has one), its size.**

There are just four storages in C:

Storage name	How you make it	Where the object lives	When the object appears	When the object dies	Initialization
auto	Local definition in function	On the stack or registers (usually)	When the declaration is reached during execution	When execution reaches the end of the block	None / initializer
alloc	*alloc(), sbrk(), mmap(), tss_create()	On the heap	When the allocation function runs	When you call free() or equivalent	None / by allocator
static	Global definition, or static definition <i>within</i> function	Data segment	When the program starts	When the program terminates	Once, before startup
thread	With _Thread_local	In the thread-local space (TLS)	When the thread starts	When the thread terminates	None / initializer

Note: you can forget about thread storage for now. You probably won't need it before your third year of study, or ever.

Exercise: how many objects of each storage type in the following program?

```
int a;
int b(int c) {
    int d = a;
    return c+d;
}
```

poll: progcl [poll link - poll results](#)

What's going on?

Don't forget: functions are objects too, expressions are objects too!

What about the following:

```
int a;
extern int b;
int c() {
    static int d = a+b;
    return d;
}
```

poll: progcl [poll link - poll results](#)

What's going on?

Remember: `extern` makes a *declaration*, not a *definition*.

17.3 Objects vs. designators

Definition:

- an *object* is a *thing* (see above)
- a **designator** is the *name* for a thing.

There are two ways to “make” objects and designators in C:

- **Declaration syntax** (e.g. "int x;")
 - can be a **definition**: new designator *and* new object
 - can be a **declaration**: *only* new designator
- **Expressions** (e.g. "3+2", "foo()")
 - these always have automatic storage.

Fundamental semantics of C:

- 1) **There may be one or more designators for an object, or none at all.**
- 2) **Types are properties of designators, not of objects.**
- 3) **Objects are just arrays of char.** You can use designators to access this array.

17.3.1 Multiple designators

After:

```
int x;  
int *p = &x;  
int* foo() { return &x; }
```

Then "x", "*p" and "*foo()" are all three designators for the same object (the object that was defined by "int x;").

17.3.2 Designators and lifetimes

What about the following program:

```
int main() {  
    int *p;  
    {  
        int x = 42;  
        p = &x;  
    }  
    printf("%d\n", *p);  
    return 0;  
}
```

What's going on?

- the lifetime of the object designated by x only extends to the end of its block.
- *p is another designator to it.
- the call to printf tries to use the 2nd designator beyond the lifetime of the object. This is incorrect, but the compiler (often) won't notice.

Remember the example from previously:

```
int* square(int x) {  
    int result = x * x;  
    return &result;  
}  
  
int main() {  
    int* x = square(2);  
    printf("hello %d\n", *x);  
    return 0;  
}
```

What's going on, with your new understanding?

- &result makes a 2nd designator for the object designated by result.
- the lifetime of the object designated by result ends when square returns.
- *x becomes a 3rd designator for the object designated by result.
- printf tries to use the 3rd designator beyond the lifetime of the object.

Remember the rule from above?



Don't use objects beyond their lifetime!

17.3.3 No designators

After:

```
int * foo() {  
    int *p = malloc(sizeof(int)); // object 1 appears here  
  
    p = malloc(sizeof(int)); // object 2 appears here  
    return p;  
}
```

The expression "`*foo()`" is a designator for the 2nd object allocated by `malloc()`. The first object *still exists* but has *no designator*.

Note: once the last designator to an object is lost, there is no way to get it back. However, the object may still continue to exist. (And take up memory!)

Exercise: how many objects and how many designators does the following program make?

```
int *foo() {  
    int *p;  
    for (int i = 0; i < 10; i++) {  
        p = malloc(sizeof(int));  
    }  
    return p;  
}
```

17.3.4 Designators of different types for the same object

After:

```
int x;  
char *p = (char*) &x;  
void* foo() { return &x; }
```

Then "`x`", "`*p`" and "`*foo()`" are all three designators for the same object (the object that was defined by "`int x;`").

However the designators have different types! That's OK.

For example, the following program is very much valid:

```

int main() {
    int x = 123;
    char *p = (char*) &x; // makes a 2nd designator
    int *y = (int*) p;     // makes a 3rd designator
    printf("%d\n", *y);    // uses the 3rd designator to access the object
                           // also prints "123"
    return 0;
}

```

17.4 Object representations

“Under” every object there is an array of char, with as many char items as the sizeof of the object.

This is call the *representation* of an object.

You can usually access this array by creating a new designator with type “reference to char” referring to the same object.

For example, this program is always valid:

```

void decompose(int x) {
    char *p = &x; // creates a designator to x with type reference to char

    for (int i = 0; i < sizeof(x); i++)
        printf("char number %d: %d\n", i, p[i]);
}

int main() {
    decompose(123456789);
    decompose(-123456789);
    return 0;
}

```

Try it out! This program *can* (but may not necessarily) print the following:

```

char number 0: 21
char number 1: -51
char number 2: 91
char number 3: 7
char number 0: -21
char number 1: 50
char number 2: -92
char number 3: -8

```

Of course: **don't access the char elements of the array representation outside of the range 0 ... sizeof-1**. Otherwise, a kitten will die.

17.4.1 Assignments are just object copies

Every time you use the syntax `x = y` (assignment) in C, the language will *copy the chars* of the representation of y and use them to overwrite the chars of the representation of x.

- If the type on the left and right do not match, either:
 - a *conversion* is possible, and the value on the right is converted to the type on the left before the copy;
 - no conversion is possible, then the compiler stops with an error message.
- The number of chars copied is *determined by the type of the designator on the left side*.

For example:

```

int    x;
int    y = x;           // copy: representation of x overwrites representation of y.
                        // sizeof(int) chars copied.

char    z = x;          // conversion of x to char, then copy to z.
                        // 1 char copied.

struct person p = x;    // error: no conversion exists (from int to struct).

int     a[10];
a = x;                  // error: no conversion exists (from int to int[]).

int     *r = &x;        // copy: representation of &x overwrites representation of r.
                        // sizeof(int*) chars copied.

int     *p;

p = r;                  // copy: representation of r overwrites representation of p.
                        // sizeof(int*) chars copied.

```

17.5 Linkage of objects, extern

Linkage answers the question “When does this and that designator (in two different places in the same source file, or in two different source files) refer to the same object?”

There are just three sorts of linkages in C:

1) **External linkage** is when the same designator declaration in different files refer to the *same object* (one object for the entire program).

2) **Internal linkage** is when the same designator declaration in different files refer to *different objects*, one per translation unit (= source file), but there can be multiple declarations in the same translation unit that refer to that local object.

3) **No linkage** is when each declaration refers to different objects.

Why does this matter?

- if you need to share a static* object between multiple source file: you need external linkage.
- if you need to share a static* object between multiple functions of the same source file, but make it invisible to other source files: you need internal linkage.

(The word “static” here refers to objects with static storage, see the section on storages earlier in this chapter).

17.5.1 How to make an object with external linkage

An object with external linkage must have a *common name* throughout all the translation units of your program. (Conversely, it is not possible to give the same object different names in different translation units.)

Say you want to make an object with name *x* that has external linkage. You do this as follows:

1. exactly 1 translation unit in your program must contain a definition for *x* in global scope without any linkage specifier (neither `extern` nor ``static``).
2. In every scope in your program where the common *x* is used:
 - you use a *declaration* for *x* using the linkage specifier "extern", e.g.
`extern int i;`
 - there must not be any global scope declaration/definition of another object also named *x* but with another linkage.

Say for example you want to create a variable `counter` with external linkage, used in two source files `count.c` and `main.c`. The definition is in `count.c`. You do this as follows:

- technically correct, works but somewhat *brittle*:
 1. write the following in `count.c`: `int counter;`
 2. write the following in `main.c`: `extern int counter;`
- (Why is this brittle? **If you ever change the type of `counter` in `count.c` but forget to do so in `main.c`, the compiler won't notice and kittens may die.**)
- preferred:
 1. write the following in `count.c`: `int counter;`
 2. write the following in `counter.h`: `extern int counter;`
 3. write the following at the beginning of both `count.c` and `main.c`:
`#include "counter.h"`



Use a header to declare your objects with external linkage!

17.5.2 How to make an object with internal linkage

An object with internal linkage has the same name within the translation unit (= source file) where it used.

Two objects with internal linkage from different translation units may reuse the same name.

Say you want to make an object with name `x` that has internal linkage. You do this as follows:

1. the *first* declaration of `x` in the translation unit must occur in the global scope with the linkage specifier `"static"`. This is also a definition.
2. There is no step 2: the declaration at step 1 is available everywhere*.

However, *if* your source file also contains more than one declaration for `x` that follow the first, but then the second or later all use `"extern"`, then all these declarations refer to that same object with internal linkage, and the program is still correct. (Despite your use of `"extern"`. Yes, this is weird.)

For example:

```
static int x;

int foo() { return x; } // refers to the common `x`

extern int x; // this is useless here, but happens to be correct.
              // refers to the same `x` as the declaration above.

void bar() { x = x + 2; } // refers to the common `x`
```

Note: the step-2-that-doesnt-exist above mentions “the declaration is available everywhere”. This is not *exactly* true, consider the following source file:

```
static int x; // definition of object `x` with internal linkage.

int foo() { return x; } // this is correct and uses the common object.

int bar(int x) { return x; } // the parameter `x` hides the global scope object `x`.

int baz(int x) {
    extern int x; // this new declaration refers to `x` in global scope!
                  // i.e. the object with internal linkage.
                  // this new declaration also shadows (hides) the parameter declaration.
    return x;
}
```

17.5.3 The double meaning of `extern`

- When you use `extern` in the *first* declaration for a name in a translation unit, two things happen:
 - this is a promise that there is a definition with external linkage elsewhere — either in the same source file or another.
 - all the following declarations/definitions for the same name in global scope are constrained to also have external linkage. **If you try to later redefine the object with internal linkage, an error will occur.**
- When you use `extern` *after the first declaration* and *the first declaration is also a definition*, two things happen:
 - the first definition decides the linkage.
 - the object may have any linkage, and the `extern` keyword will not check anything.

In other words:

- `extern` declaration, then `static` declaration/definition: **compiler error**.
- `static` declaration/definition, then `extern` declaration: no error, that's correct and the `extern` declaration is ignored.

17.5.4 The double meaning of `static`

The word `"static"` in C is reused for two different sorts of declaration. Do not confuse them!

1. `static` *inside* a function body defines an object with static storage and *no linkage*.
2. `static` *outside* a function body (in global scope) defines an object with static storage and *internal linkage*.

Example:

```

int foo() {
    static int x = 0; // no linkage, `x` is "local" to `foo`.
    return ++x;
}

int bar() {
    static int x = 0; // no linkage, `x` is "local" to `bar`.
    return ++x;
}

int main() {
    return printf("%d %d\n", foo(), bar());
}

```

This always prints "1 1": the computations in `foo()` and `bar()` always operate on different objects.

Compare with:

```

static int x = 0; // internal linkage, `x` is "shared"
int foo() {
    return ++x;
}

int bar() {
    return ++x;
}

int main() {
    return printf("%d %d\n", foo(), bar());
}

```

This may print "1 2" or "2 1". The computations in `foo()` and `bar()` operate on the same object named `x`. (The order of the two values in the `printf` output is unspecified because there is no sequence point in the argument list, so the compiler is free to choose in which order `foo` and `bar` are called. See the section above on sequence points and side effects.)

17.6 Two kinds of designators

We distinguish further between:

- **Primary designators**, *at most one per object* (perhaps zero)
 - only for non-allocated objects.
 - derived from the object's definition.
 - **decide the size** of the object.
 - **honest about mutability** - either `const` or not, and the information is accurate.
 - if there is no primary designator, then the object is always mutable.
- **Secondary designators**: everything else.
 - can lie about mutability.

For example:

```

int x = 123; // makes a primary designator "x" and an object.
int *p = &x; // "&x" is secondary designator, and so is "*p"
              // "p" (without "*") is primary designator for the reference itself.

char *r;      // "r" is a primary designator for a reference.
r = malloc(10); // "malloc(10)" is secondary designator. "*r" becomes secondary designator.
               // There is no primary designator for the allocated object.

const int y = 3; // makes a primary designator "y" and an object.
                // the object is really immutable.

y = 42;         // compiler says no: y is immutable.

int *q = (int*) &y // force make a secondary designator "*q" to "y"
                  // it lies about mutability.

*q = 123;       // hmmm... (we'll get back to it.)

```

17.6.1 Mutability

Mutability is a property of *objects*.

However the *type* of each designator also *advises* whether the compiler should check if writing to the object via the designator is valid. However, getting this information right is the responsibility of the programmer, and it's possible to get this wrong via a secondary designator!

For example, what does this program print?


```
const int x = 123; // this defines an immutable object.

int main() {
    int *y = (int*) &x;
    *y = 456;
    int z = *y;
    printf("%d\n", z);
    return 0;
}
```

poll: [progcm poll link](#) - poll results

In general:

Primary designator says ...		Object is ...
nothing (no "const")		mutable
const		immutable

Object is ...	Secondary designator is ...	Program tries to write via designator
mutable	non-const	write is allowed, everything good
mutable	const	write is disallowed (compile error), everything good
immutable	const	write is disallowed (compile error), everything good
immutable	non-const	 kittens die.



Don't write to immutable objects via mutable designators!

17.6.2 Visibility

Visibility is a property of *primary designators*.

Visibility answers the question: “Where in the source code can a designator be used?”

Everything is decided by scopes.

“Scope” is the technical word for what you commonly call “block structure”:

- there is a global scope.
- each group of braces { ... } defines an additional scope.
- scopes can be nested.
- a definition of a name within a scope can *shadow* (hide) another declaration for the name in an outer scope, *for the duration of the inner scope*.
- it's not possible to define the same name two times in the same scope.
- differences with C++:
 - the parameter list is in the same scope as the function body (they are different scopes in C++).
 - the first position in for (a; b; c) does not start its own scope (it does in C++).

Hopefully you know all this already if you ever used a programming language with [lexical scoping](#). (That is, the majority of languages in use today. But not all of them, for example the shell script language does not use lexical scoping.)

For example, the following code is valid:


```

int x = 1;           // global scope.
int foo() {
    int x = 2;       // function scope.
    while(1) {
        int x = 3;   // scope of body of "while".
        // here `x` is 3.

        if (1) {
            int x = 4; // scope of body of "if".
            x++;       // This refers to the innermost closest declaration.

            // here `x` is 5.
        }

        // here is `x` 3 again.
    }

    // here is `x` 2 again.
    ...
}
// here is `x` 1 again.

```

The following code is incorrect:

```

int foo(int x) {    // function scope
    int x;          // function scope again - double definition = invalid.
    ...
}

```

So is the following:

```

int foo() {
    int i;           // function scope
    for (int i = 0; i < 10; i++) // function scope - double definition
        printf("%d\n", i);
    ...
}

```

17.7 Summary so far

- *objects* are all the things (but not types and control structures / statements).
- objects have properties: *address*, *addressability*, *mutability*, *size*.
- *storage*: where an object lives.
 - just 4 storages: *auto*, *static*, *alloc* (and *thread*, but that's less important).
- *lifetime*: when an object lives.
 - storage decides lifetime.
 - properties never change throughout the lifetime.
- *designators* are names for objects.
 - can have more than one or none at all.
 - *types* are properties of designators, not objects.
- all objects are just arrays of `char`.
 - the values in that array are called the *representation* of the object.
 - you can use a custom designator to access this array.
 - number of elements determined by `sizeof`.
- *linkage* defines how to “share” an object between different functions or different files.
 - the words “static” and “extern” control linkage.
- two kinds of designators: *primary* and *secondary*.
 - primary designator: at most 1; secondary: the rest.
 - don't write to immutable objects! (via mutable secondary designators)
 - primary designator honest about mutability, secondary can lie.
 - *visibility* is property of primary designators, controlled by block structure.

18 Pointers, references and designators

So far we have used three different words for related things:

- “references”: in this course, also common in other programming languages.
- “pointers”: not so far in this course, but very common in C books & tutorials.
- “designators”: in this course, in every technical documentation about C compilers.

Why did we avoid the word “pointer” so far?

1. for all the explanations so far, both the words “reference” and “pointer” would have been accurate — their meaning happens to overlap in the various previous contexts. So which one to use was a matter of stylistic

choice, and we went for “reference” because that's what every other programming language also does to explain the same things.

(For example: everyone talks/writes about “passing an argument to a function by reference”, not “passing an argument to a function by pointer”).

2. for the next sections, you will see there is a subtle difference between the two words “reference” and “pointers”, and the word “pointer” is actually more complex to understand. From an educational standpoint, it is useful to use different words when talking about the same thing at different levels of complexity.

(For example, in physics people learn to work with “the newtonian model” until they start to study the “standard model”, even though everything that's true when they started learning could also have been taught using the standard model.)

What's the relationship between the three words?

- “pointer” / “reference” are related; “pointer” is the more advanced concept beyond “reference”. Everything that's conceptually true about references is also true about pointers (but not the other way around).

The two words can be used in *many different programming languages*, not just C. They can also be used in many different conversations: about algorithms, data structures, performance optimization, etc.

- “designator” is another abstraction used to talk about objects *in C specifically*, and is only used for technical conversations specifically about mutability, visibility, linkage and representations for objects in C, as described in the previous chapter.

Designators also happen to be more general than references. Every reference in C happens to be a designator, but not all designators are references.

The following sections clarify this relationship.

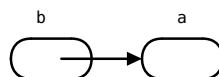
18.1 References: a logical (abstract) construct

A *reference*, in C like in other languages that have them (Java, Python...) is an object that *refers* to another object.

When we say “object b refers to object a” we mean the following abstract relationship:

- object a exists.
- object b exists. Also, it is a different object.
- there exists an *operator* which, when applied to b, designates a. This operator is called the “indirection” or “dereference”.
 - we called it `deref()` early on in this course.
 - it is written with a unary “*” in C, like this: “*b”

We can also *draw* this relationship:



Tip: when you draw these diagrams:

- place the name of the object you're talking about on the *outside* of the box.
- if you also need to talk about it, place the *value* of the object inside the box.

This way, you can talk and think separately about the name of the object and its value without getting confused between them.

Syntax-wise, suppose you have used the following declaration to define an object: `int a = 123;`

- the text “a” designates the object.
- the text “&a” makes (and designates) a new reference to the object.
- the text “`int* b`” declares+defines a variable that can contain references.
- the text “`b = &a`” (after the declaration of b) stores the reference expressed by “&a” in b.
- the text “b” (after the assignment `b = &a`) designates a reference.
- the text “*b” dereferences the reference in b, and therefore designates the object also designated by “a”.
- the text “*a” is an error (a does not designate a reference.)

- the text "`*a`" (or "`*(a)`") dereferences the reference `a`, and thus designates the same object as "`a`".

Note: this definition and relationship is abstract, and belong to the world of mathematical ideas.

Conveniently, this is also the abstraction level at which you define *data structures* and *algorithms* in this course. You could probably use only the word "reference" throughout and still get a good grade.

Warning - do not confuse the verbs:

- "*to reference*": this is a verb that you can use only after a C object. For example "this object here references that object there."
- "*to designate*": this is a verb that you can use only after a designator. For example "this name here designates that object there."

Hint: re-read the previous chapters with this distinction in mind. Check whether the two verbs were used properly.

18.2 Pointers: mechanism and implementation for references

Hint: if you are not yet comfortable with the distinction between concept and mechanism/implementation, this is the good time to give it more attention. Some examples:

- "vehicle" is a concept. The Ford T is an implementation, the design of its axle, wheels, chassis, chairs, steering wheel, motor etc. form the mechanism.
- "dog" is a concept. Fido, Pluto and Lassie are implementations. Their biological makeup form the mechanism.
- "higher education" is a concept. The BSc Informatica at UvA is an implementation. Lectures, labs and study form the mechanism.

Then on the same line:

- "references" are a concept. Pointers are an implementation of references in C; the pointer rules (detailed below) form the mechanism.

18.2.1 Pointer types

You can manipulate references in C by using objects via a designator with *pointer type*.

(Reminder: types are properties of designators, not objects.)

That is when you define `b` to be a reference:

```
int *b;
```

you are using the type "pointer to `int`" to declare `b`.

Other way to think about it: you use a pointer type when writing the declaration/definition of a primary designator as a means to achieve a declaration/definition of a reference object.

So after the declaration "`int *b`":

- the short name "`b`" is the primary designator for an object that happens to be a reference.
- the type of the designator "`b`" is "pointer to `int`".
- the expression "`*b`" designates the object referred to by `b`.

Colloquially, we say "`b` is a pointer to `a`" to mean "the object designated by `b` is a reference to `a` and the designator '`b`' has a pointer type".

18.2.2 Pointer values

Recall the definition:

```
int a;
int *b = &a;
```

This defines an object named `b` that refers to another object named `a`.

The *own value* of `b` (i.e. not the value of `a`) is also a number: it happens to be the address of `a` in memory.

Because it is "just a number" you can also "read" it! In general, if you want to *observe* the own value of a reference object (for example to print it or store it in a file), you need to either:

- first convert it explicitly to an integer type; or
- access its underlying representation as an array of `char`, as described in the previous chapter.

Example observation, by storing it in another variable with a suitable integer type:

```
int main() {
    int a;
    int *b = &a;

    long v = (long) b; // we copy the own value of `b` into `v`.
    printf("%ld\n", v); // so we can print it.
    return 0;
}
```

This program is valid, and prints some implementation-dependent value.

What's going on?

- we are using `long` in the example for the conversion instead of `int` because:
 - on many platforms, a pointer value is larger than the maximum `int` value, so it wouldn't fit in an `int` variable.
 - on most platforms, `long` happens to be large enough*.

Note: `long` is not really technically correct either. There are platforms where a pointer value does not fit in `long` either. The proper type is `intptr_t` (defined in `stdint.h`) or anything larger, for example `intmax_t`.

Another example observation, by accessing its underlying representation:

```
int main() {
    int a;
    int *b = &a; // here `b` is an object, that refers to `a`

    char *x = (char*) &b; // access the representation of `b`

    for (int i = 0; i < sizeof(b); i++)
        printf("char %d of b: %d\n", x[i]); // print all the chars
    return 0;
}
```

This program decomposes the reference in `b` to the individual `char` parts of its representation.

18.2.3 Pointer rules

A designator `x` with pointer type has an integer value and can be used as operand to the following operators:

1. `*x`

designates the object at the location in memory stored in `x`.
2. `x + N` (where `N` is an expression of integer type)

evaluates to the numeric value of `x`, added to the value of `N` times the size of `*x`. In other words: `x + N == ((intptr_t)x) + N * sizeof(*x)`

The result has pointer type again.
3. `x - N` (where `N` is an expression of integer type)

evaluates to the numeric value of `x`, subtracted with the value of `N` times the size of `*x`. In other words: `x - N == ((intptr_t)x) - N * sizeof(*x)`

The result has pointer type again.
4. `N + x` (where `N` is an expression of integer type)

equivalent to `x + N` (commutativity of `+`).
5. `x - y` (where `y` also has pointer type)

evaluates to the numeric value of `x` subtracted with the numeric value of `y`.
6. `x` cannot be used as operand to any other arithmetic operator. In particular `N - x` is invalid; `x - N` and `x - y` were really a special case.

That's all!

18.2.4 Relationship between the pointer rules and array rules

The rules above are the only rules about pointers!

Meanwhile, earlier on in a previous chapter we talked about the “array rules”, and we phrased them as follows:

1. an expression with “array type” evaluates to a reference to its first element.
2. a reference is a valid argument for a function that wants an array.
3. $a[b]$ equivalent to $*(a+b)$.
4. if a is a reference to an array element, $a+N$ is a reference to the Nth element further.

Since pointers in C are the implementation of references, this is a correct phrasing too:

1. an expression with “array type” evaluates to a reference to its first element, *and its type is a pointer type to the array element type*.
2. a ~~reference~~ *expression with pointer type* is a valid argument for a function that wants ~~an array~~ *a value with array type*.
3. $a[b]$ equivalent to $*(a+b)$. *(that remains true!)*
4. if a is a reference to an array element, $a+N$ is a reference to the Nth element further. *(that remains true too!)*

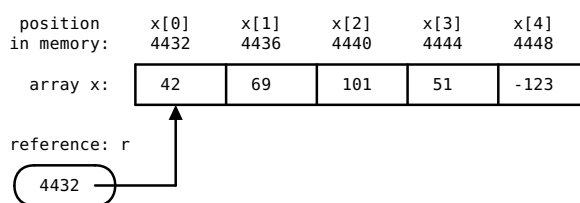
Let's rephrase rule 4: “if a is a value with pointer type, that happens to be a reference to an array element, then $a+N$ is another value with pointer type, that happens to be a reference to the Nth element further.”

Is this true?

Let's *prove* that it is true using only the pointer rules.

Consider the start situation, after you have declared the following:

```
int x[4] = { 42, 69, 101, 51, -123 };
int *r = &x;
```



Situation in this particular example:

- there is an array of 5 elements in memory.
- its 5 elements are at position 4432, 4436, 4440, 4444, 4448 in memory.
- each element takes 4 positions (bytes) in memory, because `sizeof(int) == 4`.
- the designator r has type “pointer to int”.
- “ r ” designates an object, it is a reference to $x[0]$. (an abstract relationship!)
- “ r ” designates an object, its *own value* is 4432. This happens to be the address of $x[0]$ in memory. (this is the concrete mechanism!)

Now, what happens when you write $r[2]$?

1. $r[2]$ is equivalent to $*(r + 2)$
2. We see “ $r + 2$ ”. r is a designator with pointer type, so we must use the pointer rule (2). The pointer rule says “ $r + 2$ evaluates to the numeric value of r , added to the value of 2 times the size of $*r$.”
 - r has type “pointer to int”, so the size of $*r$ is the size of `int`, i.e. 4 in this example.
 - the numeric value of r is 4432.
 - so the expression really computes $4432 + 2 * 4 = 4440$.
 - this final value in turn has pointer type.
3. The operator $*$ is applied on the result (4440) but it has pointer type, so the pointer rule (1) applies: The expression $*(r+2)$ designates the object at the location in memory computed by $r+2$, so it designates the object at location 4440, that's $x[2]$.

Tada!

18.3 Summary

- *references* are an abstract concept that you can use to mentally picture “links” to objects. You can have multiple references to the same object, you can draw little boxes with arrows to visualize them, etc.
- *pointers* are the mechanism in C to implement references.
 - you make a pointer by defining an object with *pointer type*.
 - reminder: the designator has pointer type, not the object itself!

- the object is actually just a variable that stores a number.
- the *value* of that object is a number, which may be the address in memory of an arbitrary object.
- the *pointer rules* allow you to perform basic operations on designators with pointer type.
- the phrase “pointer variable” is a colloquialism that means “variable declared with pointer type”.
- the phrase “a points to b” is a colloquialism that means “a designates an object which is a reference to b, and a was declared with pointer type”.

The person who uses this last phrase *may* also intend to mean that “the value of the object designated by a is the address of b in memory”, but usually this is not the case, because people who talk about pointers usually think about the abstract concept of reference and do not care about memory addresses that much.

19 Types, declarations and declarators

Can you read C declarations?

This is a declaration for a variable named `c`, whose designator has type `int`:

```
int c;
```

Nothing special? What about this:

```
char *(*( **foo[][8])())[];
```

19.1 Basic and derived types

Remember the chapter on [types](#) earlier in this document. It made the distinction between “basic” types (`int`, `char` etc) and “composite” types built using other types (`struct`, `arrays`, etc.)

For this chapter, we'll use a different classification:

- **Basic** types here encompass “basic” types from earlier, and also `enums`, `structs` and `unions`.
- **Derived** types encompass the rest: *array*, *function*, and *pointer* types.

These types are called “derived” because a declaration for a designator of this type includes the syntax of some basic type.

The following sections explain how to *name* a type: how you as a person can phrase the name of the type when you explain your code. They should also guide you with the opposite process: how to go from the name to the type.

Then there are three derivations possible from existing types:

- “`*`”, called “**pointer to ...**”

This is denoted by the familiar `*` character. The type being pointed to must be phrase

- “`[]`”, called “**array of ...**”

“Array of” can be undimensioned — `[]` — or dimensioned — `[10]` — but the sizes don't really play significantly into reading a declaration. We typically include the size in the description.

- “`()`”, called “**function returning ...**”

This is usually denoted by a pair of parentheses together — `()` — though it is also possible to find a prototype parameter list inside. **Important:** the parentheses used to denote “function returning” are different from those used for grouping: grouping parens surround the variable name, while “function returning” parens are always on the right of the name.

The description of a derived type *must* always include the type that is being derived. To describe a declaration, you will always use the words “to”, “of” or “returning”. For example, saying “the type of `foo` is pointer” is non-sensical.

19.1.1 Operator precedence (reminder)

Remember from section [Operators, precedence and associativity](#) there are 15 precedence levels. The particular point you need to remember here:

The postfix function application and array indexing with `()` and `[]` have a higher precedence than the prefix indirection operator `*`.

That is, an expression of the form `*p()` is to be read “`*(p())`” (not `(*p)()`!)

and `*p[i]` is `*(p[i])` (not `(*p)[i]`!).

19.1.2 A simple example

The examples in this section and the following section are borrowed from this page: <http://unixwiz.net/techtips/reading-cdecl.html>

Consider: `long **foo[7];`

General rules:

- a declaration can only have one basic type, and it is always on the far left of the declaration syntax.
- we always start the description sentence with the identifier, and end it with the basic type on the left.

For this example, the sentence thus will have the following structure: "foo designates long".

Now, starting from the name `foo`, we use the rules of operator precedence to decompose the meaning.

1. `**foo[7]` is really `*(*(foo[7]))`
2. first is `foo[7]`: "foo designates an **array of 7 ...**"
3. then `*`: "foo designates an array of 7 **pointer to ...**"
4. then `*` again: "foo designates an array of 7 pointer to **pointer to ...**"
5. nothing any more, so we end with the basic type on the left: "foo designates an array of 7 pointer to pointer to long."

19.1.3 A more complex example

Consider the example from earlier:

```
char *(*( **foo[][8])())[];
```

Applying the general principle, we know the description will be like this: "foo designates char".

Then we apply the priority rules starting from `foo`:

1. `foo[]`: "foo designates an **array of ...**"
2. `...[8]`: "foo designates an array of **array of 8* ...**"
3. `*...:` "foo designates an array of array of 8 **pointer to ...**"
4. `*...:` "foo designates an array of array of 8 pointer to **pointer to ...**"
5. `(...)( )` (note the function parentheses on the right!): "foo designates an array of array of 8 pointer to pointer to **function returning ...**"
6. `*...:` "foo designates an array of array of 8 pointer to pointer to function returning **pointer to ...**"
7. `(...)[ ]`: "foo designates an array of array of 8 pointer to pointer to function returning pointer to **array of ...**"
8. `*...:` "foo designates an array of array of 8 pointer to pointer to function returning pointer to array of **pointer to ...**"
9. nothing any more, so we finish with the basic type on the left:

"foo designates an array of array of 8 pointer to pointer to function returning pointer to array of pointer to char."

Tada! You may not *understand* what this means, but at least you can read it!

(Note: this example is artificial. Nobody in their sane mind should use this type as-is in production code.)

Tip: the following web site automates this conversion: <http://cdecl.org/>

19.1.4 Restrictions on derived types

The following rules always apply:

1. you cannot define or use a type that is an array of functions. (array of pointers to functions are OK though).
2. functions cannot return functions. (they can return pointers to functions though).
3. functions cannot return arrays. (they can return pointers though).
4. in array types, only the leftmost `[]` can omit the number of elements.
5. the pseudo-type "void" can only be used in derivation, and can only be derived using "pointer to" and "function returning", *not* "array of".

Examples:

```

int foo[](); // foo is array of function returning int: invalid (rule 1)
int (*foo[])(); // OK: foo is array of pointer to function returning int.

int foo()(); // foo is function returning function returning int: invalid (rule 2)
int (*foo())(); // OK: foo is function returning pointer to function returning int

int foo()[10]; // foo is function returning array of 10 int: invalid (rule 3)
int (*foo())[10]; // OK: foo is function returning pointer to array of 10 int

int foo[10][]; // foo is array of 10 array of int: invalid (rule 4)
int foo[][10]; // OK, but not strictly equivalent: foo is array of array of 10 int
int *foo[10]; // OK, also not equivalent: foo is array of 10 pointer to int

void foo; // invalid: void can only be used in derivations (rule 5).
void foo[10]; // foo is array of 10 void: invalid (rule 5)
void *foo; // OK: foo is pointer to void
void foo(); // OK: foo is function returning void

```

19.2 Declarations and declarators

What we have called a “declaration” so far was using a syntax construct with the basic type on the left, and the thing on the right with all the *, [] and () operators applied which can construct a derived type.

That syntax is really called a **declarator**.

What is really called a *declaration* in C is formed by using a declarator in combination with some [storage/linkage](#) specifiers:

declaration = declarator + storage + linkage, followed by a semicolon.

Of course, as seen previously the storage and linkage specifiers are optional (C will pick some storage and linkage for you if you don't choose any yourself). So all declarators followed by a semicolon (“;”) also happen to be valid declarations.

19.2.1 Declarator positions

There are just 7 places where you can use declarators in C:

- declarations/definitions in the global scope or in function scope;
- function definitions;
- function parameters;
- member definitions in struct and union;
- type alias definitions with typedef.
- as **abstract declarators**:
 - as argument to the sizeof operator;
 - in the cast operator.

Clarifications / notes:

- when you write this: `int foo(int x) { return x + 3; }`, the part of this syntax before the opening brace `int foo(int x)` is a declarator, but is not a declaration! A declaration is a declarator followed by a semicolon, but here the declarator is followed by an opening brace. A function definition is really not a declaration.
- when you write `int foo(int x)`, the part inside the parentheses is the function parameter list. Each of the items in this comma-separated list is itself a declarator. For example `int x` here is a declarator too. However again it is not a declaration, because it is not followed by a semicolon.
- also, member definitions inside a struct or union, although they are declarators followed by semicolons, are not declaration, because a declaration must appear in the global scope or in function scope. The braces around the member definitions are not a scope.

19.2.2 typedef is about defining type aliases

We have used typedef many times already but it wasn't clearly stated:

- the typedef syntax is really the keyword `typedef` followed by a declarator.
 - therefore, it may not contain any storage or linkage specifier.
- a typedef declaration defines the name inside the declarator to be equal to the type of the declarator. (This is an equality at type level!)

For example:

```

typedef char *(*(**blabla[][8]))[];
blabla foo;

```

This defines the name `blabla` to be equal to the type “array of array of 8 pointer

to pointer to function returning pointer to array of pointer to char".

The following declaration "blabla foo;" is then equivalent to the foo declaration in the example section earlier.

19.2.3 Abstract declarators

An *abstract declarator* is formed by a basic type followed by a declarator *without a name*.

Full declarator	Abstract declarator	Example cast syntax	Example sizeof syntax
int a	int	(int)x	sizeof int
int *a	int *	(int*)x	sizeof (int*)
int a[10]	int [10]	(int [10])x (note: the syntax is valid, but this cast is not valid, see note A below)	sizeof (int [10]) (ok!)
int a()	int ()	(int ())x (see note A)	sizeof (int ()) (see note B below)
int *a[10]	int *[10]	(int *[10])x (see note A)	sizeof (int *[10]) (ok!)
int (*a)[10]	int (*)[10]	(int (*)[10])x (ok!)	sizeof (int (*)[10])
int (*a)()	int (*)()	(int (*)())x	sizeof (int (*)())
int (*(a)())()	int (*(*)())()	(int (*(foo)())())x	sizeof (int (*(*)())())
char *(*a[][8])()[]	char *(*[][8])()[]	(char *(*[][8])())[]x	sizeof (char *(*[][8])())[]

Note A: it is not possible to cast an object to an array type. This will be covered in the next chapter.

Note B: a function is an object, so it is possible to apply the sizeof operator on it. However, most C compilers do not know the size of functions at compile time, yet sizeof must produce a compile-time constant, so they often pick the arbitrary value "1" as result.

19.3 Summary

- there are 3 type derivations: "array of ...", "function returning ..." and "pointer to ..."
- the construct to use a derived type is called a *declarator*.
 - every declaration includes a declarator.
- to read a declarator, start with the name, then follow operator precedence, then end with the basic type on the left.
 - remember the operator precedence: postfix binds tighter than prefix.
- derived types are restricted:
 - no array of functions (but arrays of pointers to functions are ok).
 - functions can't return arrays or functions directly (but can return pointers to them).
 - only the leftmost array derivation can omit the number of elements.
 - "void" can only be used in derivations.
 - "void" can only be used with "function returning ..." and "pointer to ...", not "array of".
- casts and sizeof use *abstract declarators*, which are simply declarators without a name.

20 Type and value conversions

A *conversion* may occur when you use a designator with a certain type in a context where another type is expected.

For example:

```
int x = 123;
double y;
y = x; // int on the right, double on the left -> conversion
```

20.1 Conversion between numeric types

Start type	New type	Value fits in new type?	Result value	Safe?	Common?
any non-struct / union	_Bool		0 if the original value compares equal to 0, 1 otherwise	😺	😺
_Bool	any non-struct / union		0 or 1	😺	😺

Some examples:

```
int zi = 0, oi = 1, ti = 42;

_Bool zb = zi; // ok: bi == 0
_Bool ob = oi; // ok: ob == 1
_Bool tb = ti; // ok: tb == 1

int zi2 = zb; // ok: zi2 == 0
int oi2 = ob; // ok: oi2 == 1
int ti2 = tb; // ok: ti2 == 1

int eb = (int)(bool)(int)42; // ok: eb == 1
```

Next batch of conversions:

Start type	New type	Value fits in new type?	Result value	Safe?	Common?
any integer / enum	any integer / enum	yes	same value	😺	😺😺😺
any integer / enum	unsigned integer	no	add original value to (max+1) of new type (causes wraparound)	😺	😺😺😺
any integer / enum	signed integer, or enum	no	either some arbitrary value or a run-time error (no dead kittens though)	😞	😞😞

Some examples:

```
int ti = -420;

signed long tl = ti; // ok: tl == -420 (-420 fits in signed long)

unsigned long tul = ti; // ok: tul == ULONG_MAX + 1 - 420

signed char tc = ti; // unpredictable, or error
// (signed char only allows -128...127)
// (however: no kittens die here.)
```

Exercise, how to check whether the conversion is possible? You can choose between:

- if ((char)ti > CHAR_MAX || (char)ti < CHAR_MIN) error()
- if (ti > (int)CHAR_MAX || ti < (int)CHAR_MIN) error()

Only one of the two is correct; the other may not do what you want it to do.

Exercise, trick question, what happens here?

```
char a[800000000L] = { 0 }; // fills array with 0

int main() {
    for (int i = 0; i < sizeof(a); i++) {
        a[i] = 42;
    }
    printf("%d\n", a[700000000L]);
    return 0;
}
```

Hint: what is the type of the result of the sizeof operator? Is it signed or

unsigned?

Next batch of conversions:

Start type	New type	Value fits in new type?	Result value	Safe?	Common?
any float	any integer / enum	integral part fits	integral part (fractional part is discarded, number rounded towards zero)	😺	😺😺😺
any float	any integer / enum	integral part does not fit	... kittens die ... (undefined behavior)	😺😺	😞😞😞

Some examples:

```
float pi = 3.14159265;

int ipi = pi; // ok: ipi == 3

float tf = 6.022140857e23;

int ti = tf; // poor kittens!!!!
            // (range of int extends to ~4e9 or ~9e18, not 6e23!)
```

Same exercise, how to check whether the conversion is possible? You can choose between:

- if ((int)tf >= INT_MAX || (int)tf <= INT_MIN) error()
- if (tf >= (float)(INT_MAX+1) || tf <= (float)(INT_MIN-1)) error()
- if (tf >= (float)INT_MAX+1.0 || tf <= (float)INT_MIN-1.0) error()

Two of these answers kill kitten!

Next batch of conversion:

Start type	New type	Value fits in new type?	Result value	Safe?	Common?
any integer / float / enum	any float	yes	same value	😺	😺😺😺
any integer / float / enum	any float	approximation fits	value of approximation	😺	😺😺😺
any integer / float / enum	any float	approximation does not fit	... kittens die ... (undefined behavior)	😺😺	😞😞😞

Check out this link about how floating point values work: <http://liv.science.uva.nl/dfp.html>

Some examples:

```
int ti = -420;
float tf = ti; // ok: tf == -42 (it fits!)

int ti2 = 100000042;
float tf2 = ti2; // ok: tf2 == 100000040
                // (approximated, but fits)

double smalld = 3.14;
float smallf = smalld; // ok: smallf == 3.14 (value fits)

double mediumd = 100000042; // (mediumd == 100000042,
                             // exact representation possible in double)
float mediumf = mediumd; // ok: mediumd == 100000040 (approximated, but fits)

double bigd = 1e200;
float bigf = bigd;      // poor kittens!
```

Final batch of conversion:

Start type	New type	Value fits in new type?	Result value	Safe?	Common?
any complex	any complex		result using separate conversion for the real and imaginary parts	😺	
any integer / float / enum	any complex		result using conversion to the real type, with imaginary part set to 0	😺	
any complex	any integer / float / enum		result using conversion of real part, imaginary part discarded	😺	

(No examples here: if you know that you need complex numbers, you're smart enough to understand what's going on here.)

You can give extra attention to the problematic cases in the tables above:

- if you convert an integer into a signed integer type and the value doesn't fit, the results are either unpredictable or a run-time error will occur.
- if you convert to/from float or double and the result does not fit the target type, **kittens will die!** (undefined behavior)

20.2 Conversion between pointer types

Start type	New type	Value fits in new type?	Result value	Safe?	Common?
pointer to void	any pointer		same value; null pointers (that compare equal to 0) are preserved	😺	😺😺😺
any pointer	pointer to void		same value; null pointers perserved	😺	😺😺😺

Some examples:

```
int a;
void *p = &a;      // ok: p == &a
char *q = (char*)p; // ok: p == q and q == &a
void *r = r;       // ok: r == q, r == p and r == &a

void *s = 0;       // ok: s is null pointer, s == 0
char *t = (char*)s; // ok: t == 0
void *u = t;       // ok: u == 0
```

Next batch of conversions:

Start type	New type	Value fits in new type?	Result value	Safe?	Common?
any pointer to non-function object	any pointer to non-function object	yes (see note below)	same value; null pointers preserved	😺	😺😺😺
any pointer to non-function object	any pointer to non-function object	no (see note below)	same value (!); null pointers preserved	😞	😞
any pointer to function	any pointer to function		same value; null pointers preserved	😞	

Note: conversion between different pointer types to object types has some restrictions that have to do with alignment, mutability, etc. The rules are

complex and not detailed here, but the rule of thumb is:

- you may get a compiler warning, however
- the conversion is possible, and
- you can store or copy the result value, or convert the result to another type, however
- if you try to use the resulting pointer in an expression with the indirection (dereference) operator, **kittens will die**.

Some examples:

```
int a;
int *p = &a;

short *q = (short*)p; // ok: q == p; here "**q" can be evaluated
long *r = (long*)p;   // ok: r == p; however here "**r" cannot be evaluated,
                      // lest kittens will die.

int *s = (int*)q;      // ok: s == q and s == p; *s refers to a.
int *t = (int*)r;      // ok: t == r and t == p; *t refers to a.

int foo() {}

int (*p)() = &foo;      // ok: p == &foo

float (*q)(double x, int y) =
    (float (*)(double, int))p; // ok: q == p and q == &foo
                              // however "(*q)()" cannot be evaluated,
                              // lest kittens will die.

int (*r)() = (int (*)())q; // ok: r == q, r == p and r == &foo
```

20.3 Mixed pointer / integer conversions

The following conversions are fraught with peril, because you can't predict their result:

Start type	New type	Value fits in new type?	Result value	Safe?	Common?
any integer / enum	any pointer	yes (see note below)	if the value was 0, then the result is a null pointer; otherwise it is implementation-defined, arbitrary, but can be converted back to integer type	☹	
any integer / enum	any pointer	no (see note below)	implementation-defined, arbitrary, cannot be converted back	☹	
any pointer	any integer / enum	yes	implementation-defined (in practice, often equal to the original value if the pointer value was initialized with an integer to start with, or 0 for null pointers)	☹	
any pointer	any integer / enum	no	... kittens die ... (undefined behavior)	😱😱	

Try to avoid them!

(They are mentioned here for completeness; you should only use them if you are absolutely sure you know what you are doing. Who does? Probably only compiler and operating system implementers.)

20.4 Pseudo-conversions

These take a value and produce a new value unrelated to the original value. (You already know one of them!)

Start type	New type	Result value
any array	any pointer	address of first element
any function	any pointer	address of function

In other words:

```
int a[10];
int *p = (int*)a;    // ok! (you know this already)

int foo();
int (*p)() = foo;    // ok!
void *q = foo;        // ok!
int *r = (int*)foo;  // ok (but beware)
```

20.5 Invalid conversions

The following conversions are guaranteed to cause compile errors, which is a good thing (the compiler detects the error for you):

Start type	New type
any	any struct / union not exactly equal to the start type
struct / union	any type not exactly equal to the start type
any pointer	any non-pointer, non-integer
any non-pointer, non-integer	any pointer
any pointer to function	any pointer to non-function object (not void)
any pointer to non-function object (not void)	any pointer to function

20.6 Automatic and explicit conversions

In many cases C will *automatically convert* between types:

- when the lvalue in an assignment has a different type than the rvalue;
- when the operands to an arithmetic operation have different types;
- when the argument provided to a function has a different type than the parameter expected by the function;
- etc.

Two points of note:

- you may disagree about the conversions to perform.

For example, by default, "3.15 < 123" will convert first 123 to float, then compare floats; you may wish to convert both to int before the comparison instead.

- in some cases the compiler will protect you by refusing to perform the conversion automatically.

For example, conversions between pointer and integer types are disallowed by default.

In the cases where you wish to perform a conversion the compiler doesn't do on its own, you can use the **cast expression**, also called **explicit conversion operator**.

You have seen this many times already! Its syntax is:

()

For example "(int)x", "(struct person*)p", or "(int (*)())q".

20.7 Summary

- conversions between types and `_Bool` is easy and uninteresting.
- beware of converting values too big to a signed integer type.
 - you may get an error or an unpredictable result.
 - be wary of unsigned → signed conversions!
 - signed → unsigned is safe though.
 - **be especially wary that many things have type `size_t` which is both unsigned and often larger than `int`.**
- conversions from float/double to integers will remove the fractional part.
 - but if the whole part is too large, kittens will die!

- conversions of numeric types to float will *approximate* the value in most cases.
 - and if the value is too large/small, kittens will die!
- conversions between pointer types and to/from the type “pointer to void” is always possible and lossless.
 - other pointer/pointer conversions need to be treated with care.
 - **don't dereference (indirect) via a pointer of incorrect type!**

otherwise kittens will die.
- mixed pointer/integer conversions are highly discouraged (but possible)
- if you don't like the default automatic conversion, or there is no automatic conversion where you want one, you can force your own conversion with the *cast operator*.
 - remember, this uses the abstract declarator syntax.

21 Killing the magic in multi-dimensional arrays

True statement: **C knows nothing about multi-dimensional arrays.**

What's true:

- C has *declarators* and a [type derivation rule](#) to make a new type “array of” another type. This can be applied recursively.
- The representation (see section [object representations](#) earlier) of an array is the concatenation of the representations of its elements.
- C has an “[array rule](#)” (really a “[pointer rule](#)”) which says that if *r* is a reference to an array element, *r+N* is a reference to the element *N* positions further.

That's all!

Now look at this example:

```
int a[10][20];

int main() {
    int i = 2, j = 3;
    printf("%d\n", a[i][j]);
    return 0;
}
```

What's going on?

1. *a* is declared as a designator with type “array of 10 array of 20 int.” (regular rules to read designators)
2. the declaration is also a definition for an object designated by *a* (declaration in global scope is a definition)
3. in the expression “*a*[*i*][*j*]” the operator precedence tells us this is really “(*a*[*i*])[*j*]” (regular precedence rule)
4. so “*a*[*i*]” is evaluated first. This accesses the *i*-th element of *a*, which is the 2nd element from the start, which is an object with type “array of 20 int”. (regular indexing operator)
5. the result of the evaluation has array type but *it is not used as rvalue* (the array indexing operator that follows is a lvalue operator), so it does *not* degrade immediately into a reference to the first element. It is still an array. (reminder: see the [note about the array rules](#) above in the chapter about pointer types.)
6. then “...[*j*]” is evaluated on this resulting array. This accesses the *j*-th element of the 20-sized array at position *i* in *a*. (regular indexing operator)
7. the result of this is an int, which can be printed.

Tada!

Note: *incidentally*, the representation of the object designated by *a* in this example is the concatenation of ...

- 10 times the representation of...
- 20 times the representation of...
- an int.

So it is really 200 ints one after another in memory.

This means that *you* can also compute the address of an arbitrary item in memory: the address of *a*[*i*][*j*] is the address of *a*[0][0] plus *i* times the size of an array of 20 ints, plus *j* times the size of an int.

In other words:

```

// two levels of arrays
// assuming N is the rightmost dimension
// assuming T is the type of 1 element
&a[i][j] == (int*)( (intptr_t)&a[0][0] + i * N * sizeof(T) + j * sizeof(T) )
           == (int*)( (intptr_t)&a[0][0] + (i*N + j) * sizeof(T) )

// three levels of arrays
// assuming N is the rightmost dimension
// assuming M is the middle dimension
// assuming T is the type of 1 element
&a[i][j][k] == (int*)( (intptr_t)&a[0][0][0] + (i*N*M + j*N + k) * sizeof(T) )

```

This formula is *sometimes* useful to reason about memory accesses but is *not necessary* at all to understand how arrays are accessed, you can simply look at steps 1-7 above.

No real need to think about “multi-dimensional” arrays. Arrays in C are single-dimensional for all practical intents and purposes.

22 Machine behavior

22.1 In a nutshell

- Execution starts with `main()`.
 - or really, `_start` on most POSIX systems.
- When `main()` returns or `exit()` is called:
 - all functions registered with `atexit()` are called in turn.
 - avoid this (if strictly necessary) by calling `_Exit()` instead.
- Execution may be arbitrarily preempted by *signal delivery*
 - Topic of next course “Operating systems”
- Program starts with just 1 thread of execution.
 - You can create more with e.g. `thr_create()` (C11) or `pthread_create()` (POSIX)
 - Topic of other course “Parallel programming”

22.2 Undefined vs Unspecified

Three parties to decide what a program means:

- the C standard (= the specification of the C language)
- the programmers of the C compiler and its standard library
- the programmers of the operating system where the program runs, or the designers of the hardware platform.

The C standard defines / *specifies* most of it.

The word “*unspecified*” when used about anything in C has a specific meaning! it means that:

- the C standard is silent about what the thing should mean, but
- the programmers of the compiler, standard library, operating system or hardware platform *have decided* something.

In other words, **if something is unspecified, you cannot know what it is by looking at the C specs, but you will know what it is for a given platform by trying it out.**

Meanwhile, the word “*undefined*” also has a specific meaning! It means that...

... you can't know.

(Also, it means that kittens die.)

22.3 Example unspecified behavior

The following aspects of the C language are *unspecified*:

- the order of evaluation of the arguments to a function.

In `foo(bar(), baz())` is `bar` evaluated before `baz`? The language spec is silent, but every compiler will choose *something*, which you can discover by running the code.
- the relative position or proximity of objects allocated by `malloc()`.
- the representation of uninitialized objects.
 - In particular you can always discover the representation (= read the bytes one after the other) of an uninitialized object by taking a `char*`

reference to it.

- However, *beware* the uninitialized representation may not encode a valid value for the type of the designator. So if e.g. you try to evaluate an int variable that wasn't initialized, you may get *undefined* behavior.

22.4 Examples undefined behavior

You've seen many already!

- exiting from a non-void function without a value.
(e.g. `int foo()`
`{}` or `int main() { printf("hello\n"); }`)
- using an object beyond its lifetime.
(e.g. use after free)
- accessing a position in memory beyond the size of an object.
(e.g. size too small for malloc, invalid array position)
- indirecting / dereferencing a null reference.
(e.g. `*p` when `p == 0`)
- writing a const object via a non-const designator.

Some are surprising however:

- performing arithmetic on two signed expressions when the result is too large for the signed type.
- reaching an infinite loop whose body does not contain side effects.

There are many more cases.

22.5 What really happens upon reaching undefined behavior?

Actual behavior	How often?
Execution stops	rare
Signal is delivered (OS says “stop”)	rare
Unrelated objects unpredictably modified	common
Execution continues from a predictable position	uncommon
Execution continues from an unpredictable position	common
If execution continues, further code changes meaning (!)	common (!)

22.6 Further reading

Try this:

- https://en.wikipedia.org/wiki/Undefined_behavior
- <http://en.cppreference.com/w/cpp/language/ub>

And about the question “Why?????”

- <http://blog.llvm.org/2011/05/what-every-c-programmer-should-know.html>
- http://blog.llvm.org/2011/05/what-every-c-programmer-should-know_14.html
- http://blog.llvm.org/2011/05/what-every-c-programmer-should-know_21.html

23 Variadic arguments

Remember, you can write this:

```
printf("hello %d\n", 123);
```

And also this:

```
printf("%s %s\n", "hello", "jane");
```

So printf appears to accept 2 arguments, and also 3 (and also more!). Yet all the functions (and function types) you have seen so far have accepted a fixed number of arguments. What gives?

That's because printf is a so-called **variadic function**.

Definition: a *variadic function* is a function whose declarator contains the ellipsis.

The ellipsis is a syntax element simply formed by three periods: "..."

For example, `printf` is really declared like this:

```
int printf(const char* fmt, ...);
```

Restrictions on variadic declarators:

- the ellipsis may only appear after a non-ellipsis parameter declarator.
- there cannot be any additional parameter declarators right of the ellipsis.

For example:

```
int printf(const char* fmt, ...); // correct
int blah(...);                  // incorrect
int bluh(int x, ..., int y);    // incorrect
```

23.1 Using variadic argument list in a function

This is outside of the scope of this course, but you can easily find this information online!

Hint: `#include <stdarg.h>` and the pseudo-type `va_list`.

Some useful links:

- https://en.wikipedia.org/wiki/Variadic_function
- <https://en.wikipedia.org/wiki/Stdarg.h>
- <http://pubs.opengroup.org/onlinepubs/9699919799/basedefs/stdarg.h.html>

24 Pot-pourri of weird things

You *will* run into the following. So... learn it!

1. The indirection operator does what its name says it does: you apply it to a designator with pointer type, and it gives you a designator for the referenced object.

Except in the one weird case:

Indirecting a designator with type “pointer to function” yields... the same operand.

For example, after the declaration `int (*p)();` the following expressions are all equivalent: `p`, `*p`, `**p`, `*** ** * ** * ** * p`

(In other words you cannot “access” the function with the indirection operator. It is still possible to call it though, see below.)

2. The “function call” operator (in expressions, not in declarators!) does what it says it does: you apply it to a designator with function type, and it calls that function.

Except in the one weird case:

A designator with type pointer to function is a valid function call expression, too.

It simply calls the referenced function.

For example, after the declaration `int (*p)();` both `(*p)()` and `p()` are equivalent ways to call the function referenced to by `p`.

This, together with the rule above, also implies the following are equivalent: `p()`, `(*p)()`, `(*** ** * ** * ** * p)()`.

3. If you do not specify a basic type for the return type of a function declarator, C picks “`int`” for you automatically (and generates a warning).

For example, `main() { ... }` is valid, and equivalent to `int main() { ... }` for this reason.

4. If you do not specify a basic type for the declarator of a function parameter, C picks “`int`” for you automatically (and generates a warning). *This is very bad style!*

For example, `int sum(x, y) { return x + y; }` is valid, and equivalent to `int sum(int x, int y) { return x + y; }` for this reason.

5. If you do not specify a parameter list in a function declarator, then you are saying to the C compiler “I am *not telling* you what this function accepts”
 - it means “this function accepts some unspecified argument types.”
 - it *does not mean* “this function is variadic.”
 - it *does not mean* “this function accepts no arguments.”

- This means that every time we were writing `"int main() { ... }"` this was really omitting the specification of the parameter list of `main`.
6. There are 4 (!) different valid declarators for the special function `main`:
- `"int main()"` (see previous point)
 - `"int main(void)"`
 - `"int main(int, char**)"`
(also equivalent to `"int main(int, char*[])"` because of array rules)
 - `"int main(int, char**, char**)"`
(also equivalent to `"int main(int, char*[], char*[])"` because of array rules)
7. Any of these is valid, but any other declarator will cause the C compiler to complain with an error.

(What they *mean* however, especially the last one, is outside the scope of this course. Go search online.)

Weird things happen in C.

25 What was not covered in this course

- the differences between ISO C89, C99, C11 and C14.
 - recommended: use C99 or C11 in your programs. In 2020, use C14.
 - <https://en.wikipedia.org/wiki/C99>
 - <https://en.wikipedia.org/wiki/C11>
 - C89 is really obsolete. Ignore *and* avoid it.
 - <http://www.open-std.org/jtc1/sc22/wg14/>
- the C preprocessor: `#define`, `#ifdef`, `#ifndef`, `#if ... #else .. #endif`, `#line`, etc.
 - https://en.wikipedia.org/wiki/C_preprocessor
 - <https://gcc.gnu.org/onlinedocs/cpp/>
- the extensive standard C library.
 - https://en.wikipedia.org/wiki/C_standard_library
- the `auto` keyword. It just says “automatic storage”. It's optional, so nobody cares about it.
- the `_Imaginary` keyword. You've seen `_Complex` already, so this should not be too complicated.
 - <http://en.cppreference.com/w/c/numeric/complex/imaginary>
- the `volatile` type qualifier.
 - https://en.wikipedia.org/wiki/Volatile_%28computer_programming%29
 - <http://en.cppreference.com/w/c/language/volatile>
- the `restrict` type qualifier.
 - <https://en.wikipedia.org/wiki/Restrict>
 - <http://en.cppreference.com/w/c/language/restrict>
 - <https://lwn.net/Articles/255364/>
- the `_Alignas` / `_Alignof` keywords / operators and general alignment considerations.
 - https://en.wikipedia.org/wiki/Data_structure_alignment
 - http://en.cppreference.com/w/c/language/_Alignof
 - http://en.cppreference.com/w/c/language/_Alignas
- the `_Generic` control structure (it's much too powerful for your own good).
 - <http://en.cppreference.com/w/c/language/generic>
 - <http://www.robertgamble.net/2012/01/c11-generic-selections.html>
- the `_Noreturn` function specifier.
 - http://en.cppreference.com/w/c/language/_Noreturn
- the `_Static_assert` keyword for declarations.
 - http://en.cppreference.com/w/c/language/_Static_assert
 - <http://www.robertgamble.net/2012/01/c11-static-assertions.html>
- the `_Atomic` keyword and memory access order in multithreaded programs.
 - <http://en.cppreference.com/w/c/language/atomic>
- the WEIRDEST THING: `setjmp` / `longjmp`.
 - <https://en.wikipedia.org/wiki/Setjmp.h>
 - <http://man7.org/linux/man-pages/man3/longjmp.3.html>
 - **don't use this. Your co-workers will hate you for it. So, so hard.**
- wide character support (this is less useful now that everyone uses UTF-8).
 - https://en.wikipedia.org/wiki/Wide_character
 - https://en.wikipedia.org/wiki/C_string_handling
- another WEIRDEST THING: trigraphs. These are obsolete.

- https://en.wikipedia.org/wiki/Digraphs_and_trigraphs
- the register keyword. It's nearly obsolete too.
- accessing functions from different programming languages from a C program.
 - or C functions from a different programming languages.
 - or using C to allow two different programming languages to talk to each other.

That's all folks!